

Détection d'Anomalies Financières par Intelligence Artificielle dans l'Audit PwC

Plateforme d'Analyse Intelligente des Transactions
Financières — Approche CRISP-DM

Présentée par :	Yousra Chaieb
Classe :	5SIGL2
Encadrant académique :	Pr. Mohamed Islem Samaali
Encadrant industriel :	M. Slim Zribi
Tutrice entreprise :	Mme. Hajer
Établissement d'accueil :	PricewaterhouseCoopers Tunisie
Année universitaire :	2025 / 2026

Soutenu le _____ devant le jury composé de : M./Mme. _____
(Président), Pr. Mohamed Islem Samaali (Encadrant)

Dédicaces

À mes parents, qui ont tout sacrifié pour que je puisse aller plus loin.

À mes enseignants d'ESPRIT, pour leur rigueur et leur générosité.

*À toute l'équipe d'Audit Analytics de PwC Tunisie,
pour leur confiance et leur encadrement bienveillant.*

*À mes amis et collègues de promotion,
compagnons de route dans cette aventure académique.*

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Mes remerciements les plus sincères vont à **Pr. Mohamed Islem Samaali**, mon encadrant académique, pour sa disponibilité, ses conseils éclairés et sa rigueur scientifique qui ont guidé l'ensemble de ce travail.

Je remercie chaleureusement **M. Slim Zribi** et **Mme. Hajer**, mes encadrants industriels chez PricewaterhouseCoopers Tunisie, pour m'avoir accordé leur confiance, pour la richesse des échanges professionnels et pour m'avoir intégré au sein de leur équipe d'Audit Analytics dans les meilleures conditions.

Ma reconnaissance va également à la **Direction d'ESPRIT** et au corps enseignant de la filière Ingénierie des Systèmes d'Information pour la qualité de la formation dispensée au cours de ces cinq années.

Enfin, je remercie ma famille pour son soutien indéfectible et ma promotion pour la bonne humeur qui a accompagné ce parcours.

Résumé

Résumé — Français

L'essor des transactions numériques génère des volumes de données financières impossibles à auditer manuellement. Ce projet de fin d'études, réalisé au sein de PricewaterhouseCoopers Tunisie, propose une **plateforme intelligente de détection d'anomalies financières** fondée sur des algorithmes d'apprentissage automatique supervisé et non supervisé, déployée sous forme de service web full-stack.

La méthodologie **CRISP-DM** structure l'ensemble du cycle de vie du projet, depuis la compréhension du domaine métier jusqu'au déploiement en production. Le jeu de données de référence contient **200 000 transactions**, dont **258 fraudes** (0,129 %, ratio 1:774), représentant un déséquilibre de classes extrême. Après exploration approfondie et ingénierie de variables (caractéristiques temporelles, comportementales et de solde), plusieurs modèles ont été entraînés et comparés : **Régression Logistique**, **Random Forest**, **XGBoost** (supervisés) et **Autoencodeur profond + Isolation Forest** (non supervisés).

Le meilleur modèle, **XGBoost avec SMOTE**, atteint un rappel de **84,6 %**, un F1-score de **83,5 %** et une PR-AUC de **86,7 %** au seuil optimal de 0,355. La plateforme de déploiement, construite avec **FastAPI 2.0** (backend Python) et **Next.js 15** (frontend React), intègre un pipeline de détection adaptatif à quatre modes, un moteur d'explicabilité (**SHAP**, **LIME**, **LLM**), la génération automatique de **rapports PDF** aux couleurs PwC, et un système d'authentification JWT avec piste d'audit complète.

Mots-clés : Détection de fraude, Anomalies financières, CRISP-DM, XGBoost, Autoencodeur, SHAP, FastAPI, Next.js, Audit IA, PwC.

Abstract

Abstract — English

The surge of digital transactions generates financial data volumes that cannot be audited manually. This final-year project, carried out at PricewaterhouseCoopers Tunisia, presents an **intelligent financial anomaly detection platform** based on supervised and unsupervised machine learning, deployed as a full-stack web service.

The **CRISP-DM** methodology structures the entire project lifecycle, from business understanding through production deployment. The reference dataset contains **200,000 transactions**, including **258 frauds** (0.129%, ratio 1:774), representing extreme class imbalance. After thorough exploratory analysis and feature engineering (temporal, behavioural, and balance-based features), several models were trained and compared: **Logistic Regression**, **Random Forest**, **XGBoost** (supervised) and a **deep Autoencoder + Isolation Forest** (unsupervised).

The best model, **XGBoost with SMOTE**, achieves a recall of **84.6%**, an F1-score of **83.5%**, and a PR-AUC of **86.7%** at the optimal threshold of 0.355. The deployment platform, built with **FastAPI 2.0** (Python backend) and **Next.js 15** (React frontend), integrates a four-mode adaptive detection pipeline, an explainability engine (**SHAP**, **LIME**, **LLM**), automatic **PDF report** generation in PwC branding, and a JWT authentication system with full audit trail.

Keywords: Fraud detection, Financial anomalies, CRISP-DM, XGBoost, Autoencoder, SHAP, FastAPI, Next.js, AI Audit, PwC.

Contents

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
1 Contexte Général du Projet	1
1.1 Introduction	1
1.2 Présentation de l'organisme d'accueil	1
1.2.1 PricewaterhouseCoopers — Vue d'ensemble	1
1.2.2 PwC Tunisie et le département Audit Analytics	2
1.2.3 Chiffres clés du Big Four	2
1.3 Cadre du projet	2
1.3.1 Problématique	2
1.3.2 Objectifs du projet	3
1.3.3 Périmètre et contraintes	3
1.4 Méthodologie CRISP-DM	4
1.4.1 Présentation générale	4
1.4.2 Application au projet	5
1.5 Plan du rapport	5
1.6 Conclusion du chapitre	5
2 Analyse et Spécification des Besoins	7
2.1 Introduction	7
2.2 Identification des acteurs	7
2.3 Cas d'utilisation	7
2.3.1 Diagramme général de cas d'utilisation	7
2.3.2 Description détaillée des cas d'utilisation principaux	9
2.4 Diagrammes de séquence	10
2.4.1 Séquence : Pipeline de prédiction complet	10
2.4.2 Séquence : Authentification JWT	11
2.5 Diagramme de classes du domaine	11
2.6 Backlog produit	12
2.7 Architecture fonctionnelle	13

2.8	Conclusion du chapitre	14
3	Business Understanding — Phase CRISP-DM 1	15
3.1	Introduction	15
3.2	Contexte métier : la fraude dans l’audit financier	15
3.2.1	Définition et typologies de la fraude	15
3.2.2	Contexte réglementaire	15
3.3	Objectifs métier et critères de succès	16
3.3.1	Objectifs métier	16
3.3.2	Justification du choix des métriques	16
3.4	Risques du projet	17
3.5	Plan de projet	17
3.6	Conclusion du chapitre	17
4	Data Understanding — Phase CRISP-DM 2	19
4.1	Introduction	19
4.2	Source des données	19
4.2.1	Le jeu de données PaySim	19
4.2.2	Description des colonnes brutes	20
4.3	Analyse exploratoire des données (EDA)	20
4.3.1	Distribution des types de transactions	20
4.3.2	Analyse temporelle	21
4.3.3	Distribution des montants	21
4.3.4	Analyse des soldes	22
4.3.5	Analyse des corrélations	22
4.3.6	Analyse de la baseline règle-métier	22
4.4	Qualité des données	23
4.5	Conclusion du chapitre	23
5	Data Preparation — Phase CRISP-DM 3	24
5.1	Introduction	24
5.2	Suppression des variables à risque de fuite	24
5.2.1	Problème de data leakage	24
5.3	Ingénierie des variables (Feature Engineering)	25
5.3.1	Variables temporelles	25
5.3.2	Variables de solde	25
5.3.3	Variables comportementales	26
5.4	Jeu de variables final	26
5.5	Découpage des données	27
5.6	Gestion du déséquilibre de classes	28

5.6.1	Le problème du déséquilibre	28
5.7	Normalisation	29
5.8	Pipeline de prétraitement — vue d'ensemble	29
5.9	Système de mapping sémantique des colonnes	29
5.10	Conclusion du chapitre	30
6	Modeling — Phase CRISP-DM 4	31
6.1	Introduction	31
6.2	Régression Logistique	31
6.2.1	Formulation mathématique	31
6.2.2	Configuration hyperparamètres	31
6.3	Random Forest	32
6.3.1	Principe	32
6.3.2	Configuration hyperparamètres	32
6.4	XGBoost	32
6.4.1	Formulation mathématique	32
6.4.2	Configuration et seuil optimal	33
6.5	Autoencodeur profond	33
6.5.1	Architecture	33
6.5.2	Fonction de perte et entraînement	34
6.5.3	Détermination du seuil de reconstruction	34
6.6	Isolation Forest	35
6.6.1	Principe	35
6.6.2	Configuration	35
6.7	Stratégie de combinaison des modèles	35
6.8	Conclusion du chapitre	36
7	Evaluation — Phase CRISP-DM 5	37
7.1	Introduction	37
7.2	Métriques d'évaluation	37
7.2.1	Définitions	37
7.2.2	Justification du choix de PR-AUC	37
7.3	Tableau comparatif des modèles	38
7.3.1	Analyse des résultats	38
7.3.2	Matrice de confusion — XGBoost	39
7.4	Analyse d'importance des variables — SHAP	39
7.5	Explicabilité locale — LIME	40
7.6	Explicabilité par LLM	40
7.7	Sélection du modèle de production	41
7.8	Conclusion du chapitre	41

8	Déploiement et Architecture Technique	42
8.1	Introduction	42
8.2	Architecture Générale de la Plateforme	42
8.2.1	Vue d'ensemble	42
8.2.2	Choix technologiques justifiés	43
8.3	Architecture Backend FastAPI	43
8.3.1	Structure des routes	43
8.3.2	Pipeline de prédiction asynchrone	44
8.3.3	Système de cache LRU	44
8.3.4	Démarrage et gestion du cycle de vie	44
8.4	Modèle de Données	45
8.4.1	Schéma de base de données	45
8.5	Architecture Frontend Next.js	46
8.5.1	Structure de l'application	46
8.5.2	Gestion de l'état et des données	46
8.5.3	Wizard d'analyse en 3 étapes	46
8.5.4	Bibliothèques UI	46
8.6	Sécurité et Authentification	47
8.6.1	Flux d'authentification JWT	47
8.6.2	Contrôle d'accès par rôle (RBAC)	47
8.6.3	Journal d'audit	47
8.7	Génération de Rapports	47
8.7.1	Rapport PDF (fpdf2)	47
8.7.2	Rapport DOCX	48
8.8	Conclusion du chapitre	48
9	Interfaces et Résultats	49
9.1	Introduction	49
9.2	Page d'Authentification	49
9.3	Tableau de Bord Principal	49
9.4	Gestion des Missions	50
9.5	Assistant d'Analyse — Étape 1 : Sélection des Données	51
9.6	Assistant d'Analyse — Étape 2 : Exécution du Pipeline	52
9.7	Tableau de Bord des Résultats	52
9.7.1	Indicateurs clés de résultats	52
9.7.2	Tableau de transactions classifiées	53
9.8	Interface d'Explication Multicouche	53
9.9	Génération et Téléchargement du Rapport	54
9.10	Page des Métriques des Modèles	55

9.11	Journal d’Audit	55
9.12	Résultats de Validation Fonctionnelle	56
9.12.1	Tests de performance sur le jeu de référence	56
9.12.2	Robustesse au schéma non standard	57
9.13	Conclusion du chapitre	57
10	Conclusion Générale et Perspectives	58
10.1	Bilan du Projet	58
10.1.1	Rappel des objectifs	58
10.1.2	Objectifs atteints	58
10.1.3	Contributions techniques	59
10.2	Limites Identifiées	59
10.2.1	Limites du modèle de détection	59
10.2.2	Limites architecturales	60
10.3	Perspectives d’Amélioration	60
10.3.1	Amélioration du modèle ML	60
10.3.2	Intégration RAG et LLM Avancé	60
10.3.3	Pipeline MLOps Complet	61
10.3.4	Conformité et Gouvernance IA	61
10.4	Réflexion Personnelle	61
10.5	Conclusion Générale	62
A	Extraits de Code Source	65
A.1	Architecture de l’AutoEncoder PyTorch	65
A.2	Détection Automatique du Mode de Prédiction	66
A.3	Mapping Sémantique des Colonnes	66
B	Données Complémentaires des Expériences	68
B.1	Statistiques Descriptives du Jeu de Données	68
B.2	Évolution de l’Entraînement de l’Autoencoder	68
B.3	Importances des Features RF et Corrélations	69
C	Glossaire Technique	70

List of Figures

1.1	Structure des lignes de services PwC — positionnement de l’Audit Analytics	1
1.2	Cycle CRISP-DM — six phases itératives	4
2.1	Diagramme de séquence — POST /api/predict	11
2.2	Architecture fonctionnelle de la plateforme — quatre couches	14
3.1	Planification du projet — 4 sprints agiles	17
6.1	Architecture de l’Autoencodeur : $14 \rightarrow 10 \rightarrow 7 \rightarrow [4] \rightarrow 7 \rightarrow 10 \rightarrow 14$, 596 paramètres	34
8.1	Architecture globale de la plateforme PwC Audit AI	42
9.1	Interface de connexion de la plateforme PwC Audit AI	49
9.2	Tableau de bord principal de la plateforme	50
9.3	Page de gestion des missions d’audit	51
9.4	Wizard d’analyse — Étape 1 : sélection du jeu de données	51
9.5	Wizard d’analyse — Étape 2 : exécution du pipeline	52
9.6	Tableau de bord des résultats d’analyse avec classifications	53
9.7	Interface d’explication multicouche d’une transaction suspecte	54
9.8	Interface de génération et téléchargement du rapport PwC	55
9.9	Page de métriques et comparaison des modèles de détection	55
9.10	Journal d’audit — traçabilité complète des actions	56
10.1	Pipeline MLOps cible pour la mise en production continue	61

List of Tables

1.1	Comparaison des quatre grands cabinets d’audit — données 2023	2
1.2	Correspondance phases CRISP-DM et chapitres du rapport	5
2.1	Acteurs du système et leurs responsabilités	7
2.2	Description détaillée — UC-05 : Lancer l’analyse	9
2.3	Description détaillée — UC-07 : Obtenir l’explication d’une transaction . .	10
2.4	Entités principales du modèle de données	12
2.5	Backlog produit — User stories prioritaires	13
3.1	Objectifs métier du projet — critères de succès	16
3.2	Analyse des risques projet	17
4.1	Description des colonnes du jeu de données brut	20
4.2	Distribution des types de transactions et taux de fraude par type	21
4.3	Corrélations de Pearson avec la variable cible <code>isFraud</code>	22
4.4	Bilan qualité du jeu de données	23
5.1	Variables supprimées pour éviter la fuite de données	24
5.2	Les 14 variables d’entrée du modèle standard	27
5.3	Composition des ensembles après découpage stratifié	27
5.4	Niveaux de confiance du mapping sémantique	30
6.1	Hyperparamètres de la Régression Logistique	31
6.2	Hyperparamètres du Random Forest	32
6.3	Hyperparamètres XGBoost	33
6.4	Hyperparamètres de l’Isolation Forest	35
7.1	Comparaison des modèles sur le jeu de test (39 fraudes parmi 30 215 transac- tions)	38
7.2	Matrice de confusion — XGB_smote (jeu de test, seuil = 0,355)	39
8.1	Justification des technologies utilisées	43
8.2	Groupes de routes de l’API REST	44
8.3	Matrice d’accès par rôle utilisateur	47
9.1	Résultats de validation fonctionnelle sur le jeu de test	56
9.2	Résultats du mapping sémantique sur schémas non standards	57
10.1	Bilan de réalisation des objectifs du projet	58

B.1	Statistiques descriptives des variables numériques (200 000 transactions) . .	68
B.2	Courbe de perte de l'Autoencoder (sélection d'époques, seed=42)	68
B.3	Top-10 importances Random Forest et corrélations avec la cible	69

1. Contexte Général du Projet

1.1 Introduction

L'audit financier constitue l'un des piliers de la confiance économique mondiale. Dans un contexte de digitalisation accélérée des échanges commerciaux, les volumes de transactions financières traitées quotidiennement par les grandes firmes d'audit dépassent largement les capacités d'analyse manuelle. La fraude financière, dont le coût global est estimé à plus de 5 % du chiffre d'affaires des entreprises selon l'ACFE [?], constitue un défi majeur pour les cabinets d'audit et leurs clients.

Ce projet de fin d'études s'inscrit dans cette problématique : concevoir et déployer une **plateforme intelligente d'analyse des transactions financières** capable de détecter automatiquement les anomalies susceptibles de révéler une fraude, en s'appuyant sur les techniques les plus avancées d'apprentissage automatique, et ce au sein de l'environnement professionnel de **PricewaterhouseCoopers (PwC) Tunisie**.

1.2 Présentation de l'organisme d'accueil

1.2.1 PricewaterhouseCoopers — Vue d'ensemble

PricewaterhouseCoopers (PwC) est l'un des quatre plus grands cabinets d'audit et de conseil au monde — le « Big Four » —, aux côtés de Deloitte, Ernst & Young et KPMG. Fondé en 1998 par la fusion de Price Waterhouse et de Coopers & Lybrand, le cabinet est aujourd'hui présent dans **151 pays** avec plus de **364 000 collaborateurs** et un chiffre d'affaires mondial supérieur à **53 milliards USD** pour l'exercice 2023 [?].

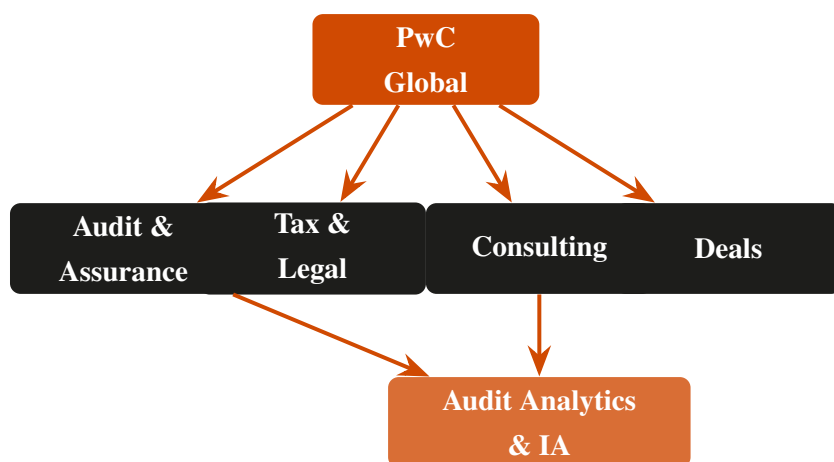


Figure 1.1: Structure des lignes de services PwC — positionnement de l'Audit Analytics

1.2.2 PwC Tunisie et le département Audit Analytics

PwC Tunisie est le bureau national du réseau PwC, implanté à Tunis. Son département **Audit Analytics** est chargé d'intégrer les nouvelles technologies — analyse de données, intelligence artificielle, visualisation — dans les missions d'audit légal et contractuel. Ce département accompagne les équipes d'audit senior dans l'analyse de jeux de données volumineux que l'approche manuelle traditionnelle ne peut traiter efficacement.

C'est au sein de ce département que s'est déroulé ce stage, dont la mission principale consistait à concevoir une plateforme de détection automatique d'anomalies dans les données de transactions financières.

1.2.3 Chiffres clés du Big Four

Table 1.1: Comparaison des quatre grands cabinets d'audit — données 2023

Cabinet	Collaborateurs	Pays	CA (Md\$)	Part marché
PwC	364 000	151	53,1	26 %
Deloitte	415 000	150	59,3	29 %
EY	365 000	150	49,4	24 %
KPMG	265 000	143	36,4	18 %
Total Big Four	1 409 000	—	198,2	97 %

1.3 Cadre du projet

1.3.1 Problématique

Les auditeurs financiers de PwC Tunisie traitent des fichiers de transactions pouvant contenir des dizaines ou des centaines de milliers de lignes. L'identification manuelle d'anomalies dans ces ensembles de données est à la fois chronophage, sujette à erreurs humaines, et statistiquement sous-optimale. Les signaux de fraude sont rares (moins de 0,2 % des transactions dans les jeux de données réalistes) et souvent noyés dans un bruit statistique considérable.

La problématique centrale de ce projet est la suivante :

Problématique centrale

Comment concevoir un système d'intelligence artificielle capable de détecter automatiquement les transactions financières anormales dans des jeux de données volumineux et fortement déséquilibrés, tout en fournissant aux auditeurs des explications compréhensibles justifiant chaque alerte ?

1.3.2 Objectifs du projet

Les objectifs opérationnels du projet se déclinent comme suit :

- O1. Détection automatique :** Entraîner et déployer des modèles de machine learning capables d'identifier les transactions frauduleuses avec un rappel supérieur à 80 %.
- O2. Gestion du déséquilibre :** Mettre en œuvre des stratégies adaptées (SMOTE, class weighting) pour traiter l'extrême déséquilibre des classes (ratio fraudes/normales = 1:774).
- O3. Explicabilité :** Intégrer des mécanismes d'explication (SHAP, LIME, LLM) permettant aux auditeurs de comprendre chaque prédiction.
- O4. Adaptabilité :** Concevoir un pipeline de détection capable de s'adapter automatiquement à des schémas de données non standards.
- O5. Déploiement :** Créer une plateforme web full-stack accessible aux équipes PwC, avec gestion des utilisateurs, des missions et des rapports.

1.3.3 Périmètre et contraintes

Le périmètre du projet couvre l'ensemble de la chaîne de valeur de la donnée : de l'ingestion du fichier CSV de transactions jusqu'à la génération du rapport d'audit PDF. Les contraintes identifiées incluent :

- La confidentialité des données réelles de PwC impose l'utilisation d'un jeu de données public synthétique (PaySim) pour l'entraînement ;
- La compatibilité avec des formats de fichiers variés et non standardisés, imposant un système de mapping sémantique de colonnes robuste ;
- La lisibilité des résultats par des auditeurs non spécialistes en data science, imposant des explications en langage naturel et des visualisations claires.

1.4 Méthodologie CRISP-DM

1.4.1 Présentation générale

La méthodologie **CRISP-DM** (Cross-Industry Standard Process for Data Mining) est adoptée comme cadre méthodologique de référence pour ce projet. Proposée en 1999 par un consortium réunissant Daimler-Benz, SPSS et NCR, elle est aujourd'hui la méthodologie la plus utilisée dans les projets de data science industriels [?].

CRISP-DM décompose un projet de fouille de données en six phases cycliques et itératives, illustrées ci-dessous :

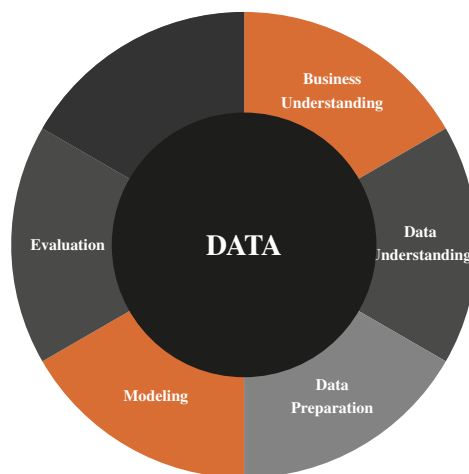


Figure 1.2: Cycle CRISP-DM — six phases itératives

1.4.2 Application au projet

Table 1.2: Correspondance phases CRISP-DM et chapitres du rapport

#	Phase CRISP-DM	Livrables dans ce projet	Chapitre
1	Business Understanding	Contexte PwC, objectifs, critères de succès	Chap. 3
2	Data Understanding	EDA, statistiques, distributions, corrélations	Chap. 4
3	Data Preparation	Nettoyage, SMOTE, encodage, normalisation	Chap. 5
4	Modeling	LR, RF, XGB, Autoencodeur, IsoForest	Chap. 6
5	Evaluation	Métriques, comparaison, sélection du modèle	Chap. 7
6	Deployment	FastAPI, Next.js, PostgreSQL, Docker	Chap. ??

1.5 Plan du rapport

Ce rapport est structuré en dix chapitres suivant une progression logique du contexte vers la solution déployée :

- **Chapitre 1** (présent) : contexte, organisme d'accueil, méthodologie.
- **Chapitre 2** : analyse et spécification des besoins (UML, backlog).
- **Chapitres 3 à 7** : déroulement intégral des phases CRISP-DM.
- **Chapitre 8** : architecture technique et déploiement.
- **Chapitre 9** : présentation des interfaces utilisateur.
- **Chapitre 10** : conclusion et perspectives.

1.6 Conclusion du chapitre

Ce premier chapitre a positionné le projet dans son environnement professionnel (PwC Audit Analytics), défini la problématique de détection automatique d'anomalies financières, et

présenté la méthodologie CRISP-DM qui structure l'ensemble du travail. Les chapitres suivants détaillent chacune des phases de cette méthodologie en s'appuyant sur les artefacts concrets produits au cours du projet.

2. Analyse et Spécification des Besoins

2.1 Introduction

L'analyse des besoins est la pierre angulaire de tout projet d'ingénierie logicielle. Elle permet de formaliser les attentes des parties prenantes sous une forme exploitable par l'équipe de développement. Ce chapitre présente les acteurs du système, les cas d'utilisation, les user stories et l'architecture fonctionnelle de la plateforme.

2.2 Identification des acteurs

La plateforme PwC de détection d'anomalies est conçue pour plusieurs profils d'utilisateurs aux responsabilités distinctes :

Table 2.1: Acteurs du système et leurs responsabilités

Acteur	Rôle	Droits principaux
Auditeur	Collaborateur en charge de l'analyse	Upload CSV, lancer analyse, consulter résultats, télécharger rapport
Manager	Responsable de mission	Toutes actions auditeur + gestion missions + consultation tableaux de bord
Partner	Associé PwC	Consultation globale, validation rapports, accès statistiques avancées
Admin	Administrateur système	Gestion complète : utilisateurs, modèles, registre, piste d'audit
Système ML	Backend FastAPI	Pipeline de prédiction, explicabilité, génération rapports

2.3 Cas d'utilisation

2.3.1 Diagramme général de cas d'utilisation

Le système offre les cas d'utilisation principaux suivants, regroupés par module fonctionnel :

2.3.1.1 Module Authentication

- **UC-01** : S'authentifier (connexion JWT)
- **UC-02** : Se déconnecter
- **UC-03** : Gérer son profil

2.3.1.2 Module Analyse

- **UC-04** : Uploader un fichier CSV de transactions
- **UC-05** : Lancer l'analyse de détection d'anomalies
- **UC-06** : Consulter les résultats de l'analyse (tableau de transactions)
- **UC-07** : Obtenir l'explication d'une transaction suspecte (SHAP/LIME/LLM)
- **UC-08** : Générer et télécharger le rapport PDF PwC

2.3.1.3 Module Gestion

- **UC-09** : Créer et gérer une mission d'audit
- **UC-10** : Consulter l'historique des analyses
- **UC-11** : Gérer les utilisateurs (admin)
- **UC-12** : Consulter la piste d'audit (admin/partner)
- **UC-13** : Gérer le registre des modèles (admin)

2.3.2 Description détaillée des cas d'utilisation principaux

Table 2.2: Description détaillée — UC-05 : Lancer l'analyse

Attribut	Description
Identifiant	UC-05
Nom	Lancer l'analyse de détection d'anomalies
Acteur principal	Auditeur, Manager
Préconditions	UC-01 réalisé ; UC-04 réalisé ou dataset existant sélectionné
Postconditions	Résultats stockés en cache (run_id), monitoring journalisé
Scénario nominal	1. L'auditeur sélectionne le fichier CSV et lance l'analyse 2. Le système mappe les colonnes sémantiquement 3. Le système sélectionne automatiquement le mode de prédiction 4. Le pipeline ML s'exécute (XGB + AE ou mode générique) 5. Les transactions sont triées par score de risque décroissant 6. Les résultats sont affichés dans le tableau de bord
Exceptions	E1 : CSV vide → HTTP 400 E2 : Colonnes non reconnues → mode dégradé (tout FAIBLE) E3 : Erreur modèle → HTTP 500 avec détail

Table 2.3: Description détaillée — UC-07 : Obtenir l’explication d’une transaction

Attribut	Description
Identifiant	UC-07
Nom	Obtenir l’explication d’une transaction suspecte
Acteur principal	Auditeur
Préconditions	UC-05 réalisé (run_id en cache)
Postconditions	Valeurs SHAP, règles LIME et texte LLM retournés
Scénario nominal	1. L’auditeur sélectionne une transaction suspecte 2. Le frontend envoie GET /api/explain?run_id=&tx_id= 3. Le backend récupère X_arr depuis le cache run_id 4. SHAP TreeExplainer calcule les valeurs SHAP XGB 5. LimeTabularExplainer génère les règles locales 6. Le LLMHelper génère un texte explicatif en français 7. L’interface affiche les trois niveaux d’explication
Exceptions	E1 : run_id expiré (cache LRU évincé) → HTTP 410 E2 : LLM indisponible → fallback rule-based activé

2.4 Diagrammes de séquence

2.4.1 Séquence : Pipeline de prédiction complet

La figure 2.1 décrit le flux d’interactions entre le frontend (navigateur), le backend FastAPI, et les services internes lors d’une analyse de détection.

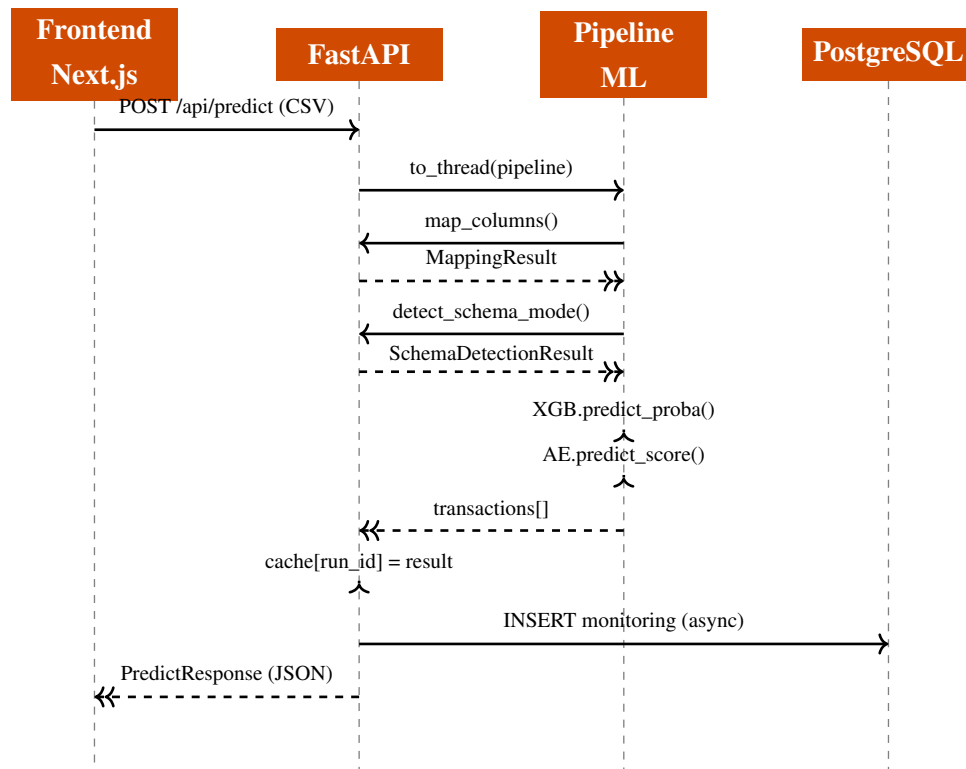


Figure 2.1: Diagramme de séquence — POST /api/predict

2.4.2 Séquence : Authentication JWT

L'authentification repose sur un token JWT signé avec la bibliothèque `jose`. Le frontend Next.js stocke le token en mémoire et l'envoie dans le header `Authorization: Bearer <token>` à chaque requête API. Côté backend, le middleware `CurrentUser` valide la signature et extrait le rôle de l'utilisateur pour contrôler l'accès aux routes sensibles.

2.5 Diagramme de classes du domaine

Le modèle de données de la plateforme est structuré en onze tables relationnelles persistées dans PostgreSQL via SQLAlchemy async. Les entités principales sont présentées dans le tableau 2.4.

Table 2.4: Entités principales du modèle de données

Table	Clé primaire	Description
users	UUID	Comptes utilisateurs avec rôle (auditor/manager/partner/admin)
missions	UUID	Missions d’audit, liées à un responsable
datasets	UUID	Métadonnées des fichiers CSV uploadés
analysis_runs	UUID	Exécutions d’analyse (run_id)
transactions	UUID	Résultats de prédiction par transaction
audit_logs	UUID	Piste d’audit complète (18 types d’actions)
reports	UUID	Rapports PDF générés
model_registry	UUID	Versions des modèles ML enregistrés
pred_monitoring	UUID	Métriques de monitoring par run
users_roles	composite	Association utilisateur-rôle
sessions	UUID	Sessions actives JWT

2.6 Backlog produit

Le backlog a été organisé selon la méthode agile en epics, features et user stories. Les éléments de priorité haute sont présentés ici.

Table 2.5: Backlog produit — User stories prioritaires

ID	User Story	Priorité	Effort	Sprint
US-01	En tant qu’auditeur, je peux uploader un CSV et obtenir les transactions suspectes classées par risque	HAUTE	8 pts	Sprint 1
US-02	En tant qu’auditeur, je peux consulter l’explication SHAP d’une transaction	HAUTE	5 pts	Sprint 1
US-03	En tant qu’auditeur, je peux générer un rapport PDF PwC en un clic	HAUTE	8 pts	Sprint 2
US-04	En tant que manager, je peux créer des missions et y rattacher des datasets	MOYENNE	5 pts	Sprint 2
US-05	En tant qu’admin, je peux consulter la piste d’audit de toutes les actions	HAUTE	3 pts	Sprint 3
US-06	En tant qu’auditeur, le système détecte automatiquement le schéma de mon fichier	HAUTE	13 pts	Sprint 1
US-07	En tant qu’auditeur, je reçois une explication en langage naturel (LLM)	MOYENNE	8 pts	Sprint 3
US-08	En tant qu’admin, je peux gérer le registre des modèles et promouvoir un modèle en production	MOYENNE	5 pts	Sprint 4

2.7 Architecture fonctionnelle

L’architecture fonctionnelle de la plateforme s’articule autour de quatre couches principales, comme illustré dans la figure 2.2 :

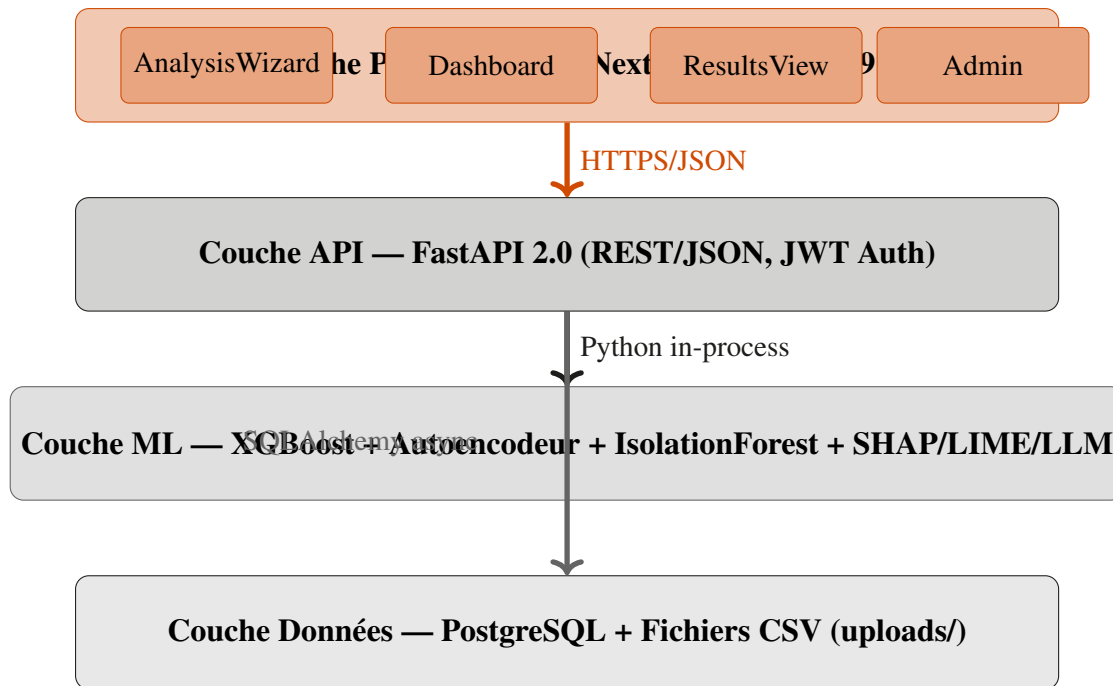


Figure 2.2: Architecture fonctionnelle de la plateforme — quatre couches

2.8 Conclusion du chapitre

Ce chapitre a formalisé les besoins de la plateforme à travers l'identification des acteurs, les cas d'utilisation, les diagrammes de séquence et le backlog produit. Ces spécifications constituent la base contractuelle du développement. Les chapitres suivants décrivent le déroulement des phases CRISP-DM depuis la compréhension métier jusqu'au déploiement.

3. Business Understanding — Phase CRISP-DM 1

3.1 Introduction

La première phase de CRISP-DM, le *Business Understanding*, vise à définir les objectifs métier du projet, à les traduire en objectifs data science mesurables, et à évaluer la situation initiale. Cette phase est fondamentale car elle garantit l'alignement entre la valeur attendue par PwC et les choix techniques qui suivront.

3.2 Contexte métier : la fraude dans l'audit financier

3.2.1 Définition et typologies de la fraude

La fraude financière se définit comme tout acte délibéré commis pour obtenir un avantage illégitime par des moyens trompeurs. Dans le cadre de l'audit financier, on distingue principalement :

- **Fraude par détournement d'actifs** : vol ou utilisation inappropriée d'actifs de l'entreprise (la forme la plus fréquente, 86 % des cas selon l'ACFE) ;
- **Fraude aux états financiers** : falsification délibérée des documents comptables pour améliorer artificiellement l'apparence financière de l'entité ;
- **Corruption et pots-de-vin** : comportements frauduleux impliquant des acteurs internes et externes à l'entreprise.

Dans le contexte des transactions de paiement électronique, la fraude se manifeste principalement sous deux formes : les **virements non autorisés** (TRANSFER) et les **retraits frauduleux** (CASH_OUT), souvent précédés d'une augmentation anormale du solde destinataire suivi d'un vidage immédiat.

3.2.2 Contexte réglementaire

Les normes d'audit internationales, notamment la norme **ISA 240** (*The Auditor's Responsibilities Relating to Fraud in an Audit of Financial Statements*), imposent aux auditeurs d'identifier et d'évaluer les risques de fraude matérielle dans les états financiers. L'utilisation d'outils d'analyse assistée par ordinateur (*Computer-Assisted Audit Techniques*, CAAT) est

explicitement encouragée pour améliorer la couverture d’audit sur des volumes de transactions importants.

3.3 Objectifs métier et critères de succès

3.3.1 Objectifs métier

Table 3.1: Objectifs métier du projet — critères de succès

Objectif métier	Traduction data science	Critère de succès
Détecter un maximum de fraudes (pas de faux négatifs)	Maximiser le rappel (Recall)	$\text{Recall} \geq 80 \%$ sur jeu de test
Minimiser les faux positifs (charge de travail auditeur)	Maximiser la précision (Precision)	$\text{Precision} \geq 70 \%$
Évaluer la performance globale sur données déséquilibrées	Maximiser F1-score et PR-AUC	$\text{F1} \geq 75 \%$, $\text{PR-AUC} \geq 75 \%$
Fournir des explications compréhensibles	Intégrer SHAP + LLM	Explication disponible pour chaque alerte
Traiter des schémas variés sans reconfiguration	Pipeline adaptatif	4 modes couvrant 100 % des cas

3.3.2 Justification du choix des métriques

Dans un problème de détection de fraude avec déséquilibre extrême (ratio 1:774), la métrique standard d’exactitude (*Accuracy*) est totalement inadaptée : un classifieur prédisant toujours «non-fraude» atteindrait 99,87 % d’exactitude tout en détectant zéro fraude. Nous privilégions donc :

- **Rappel** (*Recall* ou sensibilité) : fraction des fraudes réelles correctement identifiées. C’est la métrique prioritaire car un faux négatif (fraude manquée) est plus coûteux qu’un faux positif (transaction normale incorrectement signalée) ;

- **PR-AUC** (aire sous la courbe Précision-Rappel) : mesure robuste aux déséquilibre de classes, plus informative que la ROC-AUC dans ce contexte [?] ;
- **F1-score** : moyenne harmonique précision-rappel, offrant un compromis équilibré entre les deux.

3.4 Risques du projet

Table 3.2: Analyse des risques projet

Risque	Probabilité	Impact	Mitigation
Déséquilibre extrême des classes	Haute	Critique	SMOTE + class_weight='balanced'
Données réelles confidentielles	Haute	Élevé	Utilisation de PaySim (dataset public)
Généralisation à des schémas variés	Moyenne	Élevé	Mapping sémantique + 4 modes adaptatifs
LLM indisponible en production	Moyenne	Moyen	Fallback rule-based intégré
Surapprentissage des modèles	Moyenne	Élevé	Validation croisée + early stopping

3.5 Plan de projet

Le projet a été conduit selon une approche itérative en quatre sprints de deux semaines, couvrant l’ensemble de la chaîne CRISP-DM.

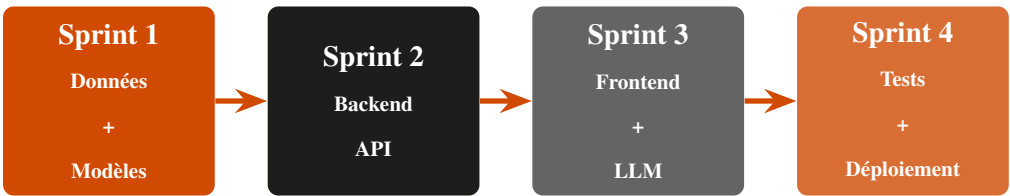


Figure 3.1: Planification du projet — 4 sprints agiles

3.6 Conclusion du chapitre

Cette phase de Business Understanding a permis de cerner précisément les enjeux métier de PwC, de définir des critères de succès quantitatifs et de cartographier les risques du projet.

Ces fondations orientent les choix méthodologiques et techniques décrits dans les chapitres suivants.

4. Data Understanding — Phase CRISP-DM 2

4.1 Introduction

La phase de *Data Understanding* constitue l'exploration approfondie des données disponibles : leur source, leur structure, leur qualité et leurs propriétés statistiques. Elle est indispensable pour orienter les choix de prétraitement et de modélisation.

4.2 Source des données

4.2.1 Le jeu de données PaySim

Le jeu de données utilisé est une version adaptée de **PaySim** [?], un simulateur de transactions financières mobiles basé sur des données agrégées réelles d'un opérateur de paiement africain. PaySim génère des transactions synthétiques reproduisant le comportement normal des utilisateurs ainsi que des comportements frauduleux injectés artificiellement.

Caractéristiques du jeu de données

- **200 000 transactions** au total
- **258 fraudes** (0,129 %, ratio fraude/normal = 1:774)
- **11 colonnes brutes** d'entrée
- **Période simulée** : 744 heures (31 jours)
- **Taille fichier** : $\approx$ 18 Mo (CSV)

4.2.2 Description des colonnes brutes

Table 4.1: Description des colonnes du jeu de données brut

Colonne	Type	Description
step	Entier	Heure de la transaction (1 à 744)
type	Catég.	Type : CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER
amount	Flottant	Montant de la transaction en devise locale
nameOrig	Chaîne	Identifiant du compte émetteur
oldbalanceOrg	Flottant	Solde émetteur avant transaction
newbalanceOrig	Flottant	Solde émetteur après transaction
nameDest	Chaîne	Identifiant du compte destinataire
oldbalanceDest	Flottant	Solde destinataire avant transaction
newbalanceDest	Flottant	Solde destinataire après transaction
isFraud	Binaire	Étiquette de vérité terrain (1=fraude, 0=normal)
isFlaggedFraud	Binaire	Système de règles interne (baseline règle-métier)

4.3 Analyse exploratoire des données (EDA)

4.3.1 Distribution des types de transactions

L'analyse de la répartition des types de transactions révèle une prédominance des paiements (PAYMENT) et des retraits (CASH_OUT), comme présenté dans le tableau 4.2.

Table 4.2: Distribution des types de transactions et taux de fraude par type

Type	Nb transactions	Part (%)	Nb fraudes	Taux fraude (%)
CASH_IN	35 241	17,6	0	0,000
CASH_OUT	66 627	33,3	127	0,191
DEBIT	1 399	0,7	0	0,000
PAYMENT	79 780	39,9	0	0,000
TRANSFER	16 953	8,5	131	0,773
Total	200 000	100,0	258	0,129

Note

Observation clé : **la fraude ne se produit que dans les transactions TRANSFER et CASH_OUT**. Toutes les transactions CASH_IN, DEBIT et PAYMENT sont légitimes dans ce jeu de données. Cette observation oriente directement le feature engineering (variable indicatrice `is_transfer_or_cashout`).

4.3.2 Analyse temporelle

L'analyse de la distribution temporelle des fraudes en fonction de l'heure de la journée révèle des schémas caractéristiques :

- **Heures à risque élevé (0h–9h et 23h)** : taux de fraude de 0,73 %, soit plus de dix fois le taux moyen. Les fraudeurs opèrent préférentiellement pendant les heures creuses où la surveillance est réduite.
- **Heures ouvrées (9h–17h)** : taux de fraude de seulement 0,07 %, cohérent avec une surveillance accrue pendant les heures de bureau.

Cette observation justifie la création de la variable `high_risk_hour` (1 si heure $\in \{0, 1, \dots, 9, 23\}$, 0 sinon), qui s'avèrera l'une des variables les plus discriminantes.

4.3.3 Distribution des montants

Les montants frauduleux présentent une distribution significativement différente de celle des transactions normales :

- **Médiane fraude** : 3 924 514 unités (montants très élevés) ;
- **Médiane normale** : 58 261 unités ;

- **Maximum observé** : 10 000 000 unités (limite système).

La transformation logarithmique $\log(1 + \text{amount})$ est appliquée pour réduire l'asymétrie et améliorer les performances des modèles à base de distance.

4.3.4 Analyse des soldes

Une caractéristique remarquable des transactions frauduleuses est le **vidage total du solde de l'émetteur** : dans 93 % des fraudes TRANSFER, le solde de l'émetteur atteint 0 après la transaction (`newbalanceOrig` = 0), contre seulement 0,8 % des transactions normales.

De même, **85 % des fraudes CASH_OUT** sont dirigées vers des comptes dont le solde initial est nul (`oldbalanceDest` = 0), suggérant l'utilisation de comptes mules ouverts spécifiquement pour recevoir des fonds frauduleux.

Ces observations motivent la création des variables `balance_diff_orig` (différence de solde émetteur) et `dest_zero_balance` (solde destinataire nul).

4.3.5 Analyse des corrélations

Table 4.3: Corrélations de Pearson avec la variable cible `isFraud`

Variable	Corrélation	Interprétation
<code>balance_diff_orig</code>	+0,366	Vidage du solde émetteur
<code>dest_zero_balance</code>	+0,109	Compte mule (solde nul)
<code>amount</code>	+0,076	Montants élevés
<code>high_risk_hour</code>	+0,053	Heures nocturnes
<code>is_transfer_or_cashout</code>	+0,037	Types de tx frauduleux
<code>type_PAYMENT</code>	-0,024	Jamais frauduleux

4.3.6 Analyse de la baseline règle-métier

Le jeu de données inclut une variable `isFlaggedFraud` représentant le système de règles heuristiques existant. Son évaluation sur le jeu de test révèle des performances catastrophiques :

- **Rappel** : 0,39 % (ne détecte qu'1 fraude sur 258) ;
- **F1-score** : 0,77 % ;
- **Seuil implicite** : montant > 200 000 unités (trop restrictif).

Cette baseline illustre l'inadéquation des approches par règles pour les fraudes sophistiquées et justifie pleinement le recours à l'apprentissage automatique.

4.4 Qualité des données

Table 4.4: Bilan qualité du jeu de données

Indicateur	Valeur	Action requise
Valeurs manquantes	0	Aucune
Doublons	0	Aucune
Colonnes à fuite de données	4	Suppression (nameOrig, nameDest, isFlaggedFraud, soldes bruts)
Déséquilibre fraude/normal	1:774	SMOTE + class_weight
Asymétrie montants	forte	Transformation log

4.5 Conclusion du chapitre

L'analyse exploratoire a mis en évidence les caractéristiques fondamentales du jeu de données : un déséquilibre de classes extrême, une fraude concentrée sur deux types de transactions (TRANSFER et CASH_OUT), des patterns temporels nocturnes prononcés, et des signatures de soldes très discriminantes. Ces observations guident directement la phase de préparation des données présentée au chapitre suivant.

5. Data Preparation — Phase CRISP-DM

3

5.1 Introduction

La préparation des données est la phase la plus chronophage de tout projet de data science, représentant typiquement 60 à 80 % de l'effort total [?]. Elle comprend le nettoyage, la suppression des fuites de données, le découpage train/validation/test, l'encodage, la normalisation, la génération de nouvelles variables et le rééchantillonnage.

5.2 Suppression des variables à risque de fuite

5.2.1 Problème de data leakage

La fuite de données (*data leakage*) se produit lorsque des informations du jeu de test contaminent l'entraînement, donnant l'illusion de performances élevées qui ne se reproduisent pas en production. Dans ce jeu de données, plusieurs variables doivent être impérativement supprimées :

Table 5.1: Variables supprimées pour éviter la fuite de données

Variable	Raison de suppression	Type de fuite
nameOrig	Identifiant client — surapprentissage	Identifiant direct
nameDest	Identifiant destinataire — surapprentissage	Identifiant direct
isFlaggedFraud	Règle métier triviale, peu informative	Variable proxy
oldbalanceOrig	Redondant avec <code>balance_diff_orig</code>	Fuite structurelle
newbalanceOrig	Calculable depuis montant + solde initial	Fuite structurelle
oldbalanceDest	Redondant	Fuite structurelle
newbalanceDest	Calculable	Fuite structurelle

5.3 Ingénierie des variables (Feature Engineering)

L'ingénierie des variables est l'une des contributions majeures de ce projet. Trois familles de variables dérivées ont été construites, toutes préfixées par `eng_` pour les distinguer des variables brutes :

5.3.1 Variables temporelles

À partir de la variable `step` (heure absolue depuis le début de la simulation, de 1 à 744) :

$$\text{hour} = (\text{step} - 1) \bmod 24 \quad (5.1)$$

$$\text{day} = \lfloor (\text{step} - 1) / 24 \rfloor \bmod 7 + 1 \quad (5.2)$$

$$\text{week} = \lfloor (\text{step} - 1) / 168 \rfloor + 1 \quad (5.3)$$

- `eng_is_weekend` : 1 si `day` $\in \{6, 7\}$, 0 sinon ;
- `eng_is_business_hour` : 1 si `hour` $\in [9, 17]$, 0 sinon ;
- `eng_tx_day_of_week` : jour de la semaine (1–7).

5.3.2 Variables de solde

Ces variables capturent la dynamique des soldes, particulièrement discriminante pour les fraudes :

$$\text{balance_diff_orig} = \text{oldbalanceOrg} - \text{newbalanceOrig} \quad (5.4)$$

$$\text{eng_drain_pct_src} = \frac{\text{amount}}{\max(\text{oldbalanceOrg}, 1)} \quad (5.5)$$

$$\text{eng_amount_ratio_src} = \frac{\text{amount}}{\max(\text{oldbalanceOrg} + \text{amount}, 1)} \quad (5.6)$$

$$\text{eng_balance_gap} = \text{oldbalanceOrg} - \text{amount} - \text{newbalanceOrig} \quad (5.7)$$

La variable `eng_balance_gap` (équation 5.7) est particulièrement révélatrice : elle mesure l'incohérence comptable de la transaction. Pour une transaction légitime, cette valeur devrait être proche de zéro ; pour une fraude impliquant de fausses écritures comptables, elle peut prendre des valeurs significativement non nulles.

5.3.3 Variables comportementales

Ces variables agrègent le comportement historique de chaque compte émetteur sur le batch courant :

- `eng_orig_tx_count` : nombre total de transactions du compte émetteur ;
- `eng_orig_unique_dests` : nombre de destinataires uniques ;
- `eng_orig_avg_amount` : montant moyen des transactions ;
- `eng_orig_total_amount` : montant total engagé ;
- `eng_dest_tx_count` : nombre de réceptions du destinataire ;
- `eng_dest_avg_received` : montant moyen reçu ;
- `eng_orig_is_high_freq` : 1 si le compte émetteur appartient aux 5 % des comptes les plus actifs (top 5 % de fréquence).

5.4 Jeu de variables final

Après ingénierie et sélection, le modèle XGBoost est entraîné sur un vecteur de **14 variables** :

Table 5.2: Les 14 variables d'entrée du modèle standard

#	Variable	Type	Description
1	log_amount	Continue	$\log(1 + \text{amount})$
2	step	Continue	Heure absolue (1–744)
3	hour	Continue	Heure dans la journée (0–23)
4	day	Continue	Jour de la semaine (1–7)
5	week	Continue	Semaine du mois (1–4)
6	balance_diff_orig	Continue	Variation de solde émetteur
7	high_risk_hour	Binaire	Heure à risque (0h–9h ou 23h)
8	is_transfer_or_cashout	Binaire	Type fraudogène
9	dest_zero_balance	Binaire	Solde destinataire nul
10	type_CASH_IN	Binaire	One-hot encodé
11	type_CASH_OUT	Binaire	One-hot encodé
12	type_DEBIT	Binaire	One-hot encodé
13	type_PAYMENT	Binaire	One-hot encodé
14	type_TRANSFER	Binaire	One-hot encodé

5.5 Découpage des données

Le jeu de données est divisé en trois ensembles selon une stratification sur la variable cible afin de maintenir le taux de fraude (0,129 %) dans chaque sous-ensemble :

$$\text{Train} : \text{Validation} : \text{Test} = 70 \% : 15 \% : 15 \% \quad (5.8)$$

Table 5.3: Composition des ensembles après découpage stratifié

Ensemble	Transactions	Fraudes	Taux fraude
Train	139 818	181	0,129 %
Validation	29 967	38	0,127 %
Test	30 215	39	0,129 %
Total	200 000	258	0,129 %

Note

Règle de non-contamination : le `StandardScaler` est ajusté (*fit*) *uniquement* sur l'ensemble d'entraînement, puis `transform` est appliqué sur les ensembles de validation et de test. Cette règle prévient la fuite d'information statistique des données de test vers le processus d'entraînement.

5.6 Gestion du déséquilibre de classes

5.6.1 Le problème du déséquilibre

Avec un ratio de 1:774 (181 fraudes pour 139 818 transactions normales dans l'ensemble d'entraînement), les classifieurs standards sont fortement biaisés vers la classe majoritaire. Deux stratégies complémentaires ont été évaluées :

5.6.1.1 Stratégie 1 : Pondération des classes (`class_weight='balanced'`)

La pondération attribue automatiquement un poids inversement proportionnel à la fréquence de chaque classe :

$$w_c = \frac{N}{K \cdot n_c} \quad (5.9)$$

où N est le nombre total d'exemples, K le nombre de classes et n_c le nombre d'exemples de la classe c . Pour nos données : $w_{\text{fraude}} = \frac{140\,000}{2 \times 181} \approx 386$.

5.6.1.2 Stratégie 2 : SMOTE — Synthetic Minority Over-sampling Technique

SMOTE [1] génère des exemples synthétiques de la classe minoritaire en interpolant entre des voisins proches existants :

$$\mathbf{x}_{\text{syn}} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_{knn} - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0, 1) \quad (5.10)$$

où $\mathbf{x}_i$ est un exemple de fraude existant, $\mathbf{x}_{knn}$ son k -ème voisin le plus proche dans la classe minoritaire, et λ un scalaire aléatoire uniforme.

Important : application de SMOTE

SMOTE est appliqué **exclusivement sur l'ensemble d'entraînement**, après le découpage train/test. L'appliquer avant le découpage constituerait une fuite de données (contamination des ensembles d'évaluation par des exemples synthétiques).

Après SMOTE, le jeu d'entraînement contient **139 637 transactions normales** et **139 637 fraudes synthétiques** — un ratio parfaitement équilibré (1:1), utilisé pour l'entraînement des

modèles XGBoost, Random Forest et Régression Logistique.

5.7 Normalisation

Le `StandardScaler` de scikit-learn est appliqué sur les variables continues afin de centrer-réduire les distributions :

$$x_{\text{scaled}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (5.11)$$

Les variables concernées sont : `step`, `hour`, `day`, `week`, `log_amount`, `balance_diff_orig`. Les variables binaires (one-hot, indicatrices) ne sont pas normalisées.

5.8 Pipeline de prétraitement — vue d'ensemble

Algorithm 1 Pipeline de prétraitement complet

Require: DataFrame brut D avec colonnes PaySim

Ensure: $(X_{\text{train}}, y_{\text{train}})_{\text{SMOTE}}$, $(X_{\text{val}}, y_{\text{val}})$, $(X_{\text{test}}, y_{\text{test}})$, scaler ajusté

- 1: Supprimer les colonnes à fuite : `COLS_TO_DROP`
 - 2: Calculer $\log_amount \leftarrow \log(1 + D[\text{amount}])$
 - 3: Calculer `balance_diff_orig`, `high_risk_hour`, `dest_zero_balance`, `is_transfer_or_cashout`
 - 4: Décoder les heures/jours/semaines depuis `step`
 - 5: One-hot encoder `type` $\rightarrow$ 5 colonnes binaires
 - 6: Découpage stratifié : $D \rightarrow D_{\text{train}}, D_{\text{val}}, D_{\text{test}}$
 - 7: Ajuster `StandardScaler` sur $D_{\text{train}}[\text{SCALE_COLS}]$ uniquement
 - 8: Transformer D_{val} et D_{test} avec le scaler ajusté
 - 9: Appliquer SMOTE sur $D_{\text{train}} \rightarrow (X_{\text{train_smote}}, y_{\text{train_smote}})$
 - 10: **return** $(X_{\text{train_smote}}, y_{\text{train_smote}})$, $(X_{\text{val}}, y_{\text{val}})$, $(X_{\text{test}}, y_{\text{test}})$, scaler
-

5.9 Système de mapping sémantique des colonnes

Un défi majeur de la plateforme est sa capacité à traiter des fichiers CSV avec des schémas de colonnes non standards (noms en français, abbréviations, etc.). Le module `ColumnMapper` (fichier `app/services/column_mapper.py`) implémente un système de matching à quatre niveaux de confiance :

Table 5.4: Niveaux de confiance du mapping sémantique

Niveau	Confiance	Exemple
Exact alias	1,00	amount → amount
Normalisé	0,95	Amount → amount
Signal composé	0,80	tx_amount → amount
Fuzzy (SequenceMatcher)	0,60	montant → amount

L’assignation est gloutonné (*greedy*) : les paires de confiance maximale sont assignées en premier, garantissant une cohérence globale du mapping. Le type de transaction est normalisé via une table d’équivalence (TYPE_VALUE_MAP) couvrant les alias français et anglais : virement → TRANSFER, retrait → CASH_OUT, paiement → PAYMENT, etc.

5.10 Conclusion du chapitre

La phase de préparation des données a produit un jeu de données propre, enrichi de 14 variables discriminantes, correctement découpé en trois ensembles stratifiés et équilibré par SMOTE pour l’entraînement. Ces données prêtes à l’emploi constituent le socle sur lequel les modèles présentés au chapitre suivant sont construits.

6. Modeling — Phase CRISP-DM 4

6.1 Introduction

La phase de modélisation constitue le cœur technique du projet. Cinq familles d’algorithmes ont été explorées, couvrant à la fois l’apprentissage supervisé (où l’étiquette de fraude est connue) et l’apprentissage non supervisé (où les anomalies sont identifiées sans étiquette). Cette approche hybride permet de couvrir des cas d’usage en production où les données étiquetées peuvent être absentes ou insuffisantes.

6.2 Régression Logistique

6.2.1 Formulation mathématique

La régression logistique modélise la probabilité d’appartenance à la classe fraude par la fonction sigmoïde appliquée à une combinaison linéaire des features :

$$P(\hat{y} = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (6.1)$$

Les paramètres $\mathbf{w}$ et b sont estimés par maximisation de la vraisemblance régularisée L2 (régularisation de Tikhonov) :

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] + \frac{\lambda}{2} \|\mathbf{w}\|^2 \quad (6.2)$$

6.2.2 Configuration hyperparamètres

Table 6.1: Hyperparamètres de la Régression Logistique

Paramètre	Valeur	Justification
C (inverse de λ)	0,1	Régularisation forte pour données déséquilibrées
max_iter	1 000	Convergence assurée avec solver lbfgs
solver	lbfgs	Efficace pour régularisation L2, grandes dimensions
class_weight	balanced	Pondération automatique des classes
random_state	42	Reproductibilité

6.3 Random Forest

6.3.1 Principe

Le Random Forest est un ensemble de T arbres de décision entraînés indépendamment sur des sous-ensembles aléatoires des données (*bagging*) et des features (*random subspace method*). La prédiction finale est la moyenne des probabilités individuelles :

$$P_{\text{RF}}(\hat{y} = 1 \mid \mathbf{x}) = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x}) \quad (6.3)$$

où $h_t(\mathbf{x})$ est la prédiction de l'arbre t .

6.3.2 Configuration hyperparamètres

Table 6.2: Hyperparamètres du Random Forest

Paramètre	Valeur	Justification
<code>n_estimators</code>	300	Bonne convergence, temps acceptable
<code>max_depth</code>	10	Limite le surapprentissage
<code>min_samples_leaf</code>	5	Feuilles suffisamment peuplées
<code>class_weight</code>	balanced	Compensation du déséquilibre
<code>n_jobs</code>	-1	Parallélisation sur tous les cœurs CPU
<code>random_state</code>	42	Reproductibilité

6.4 XGBoost

6.4.1 Formulation mathématique

XGBoost (*eXtreme Gradient Boosting*) [2] est un algorithme de boosting qui entraîne des arbres de façon séquentielle, chaque arbre corrigeant les erreurs du précédent. L'objectif est la minimisation de :

$$\mathcal{L}^{(t)} = \sum_{i=1}^N l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (6.4)$$

où l est la perte logistique, f_t le t -ième arbre, et Ω un terme de régularisation contrôlant la complexité de l'arbre :

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2 \quad (6.5)$$

avec T le nombre de feuilles et $\mathbf{w}$ les scores des feuilles.

6.4.2 Configuration et seuil optimal

Table 6.3: Hyperparamètres XGBoost

Paramètre	Valeur	Justification
<code>n_estimators</code>	300	Convergence du boosting
<code>max_depth</code>	6	Profondeur modérée
<code>learning_rate</code>	0,05	Faible taux d'apprentissage
<code>subsample</code>	0,8	Bagging des instances
<code>colsample_bytree</code>	0,8	Bagging des features
<code>scale_pos_weight</code>	774	$N_{\text{négatif}}/N_{\text{positif}}$ (non-SMOTE)
<code>eval_metric</code>	<code>aucpr</code>	Optimise directement la PR-AUC

Le seuil de décision optimal τ^* est déterminé sur l'ensemble de validation par maximisation du F1-score :

$$\tau^* = \arg\max_{\tau \in [0,1]} F_1(\tau) = \tau \frac{2 \cdot P(\tau) \cdot R(\tau)}{P(\tau) + R(\tau)} \quad (6.6)$$

La valeur retenue est $\tau^* = 0,355$, significativement inférieure au seuil par défaut de 0,5 — cohérent avec la nécessité de favoriser le rappel dans un contexte de fraude.

6.5 Autoencodeur profond

6.5.1 Architecture

L'autoencodeur (*Autoencoder*, AE) est un réseau de neurones non supervisé entraîné à reconstruire fidèlement ses entrées [?]. L'hypothèse sous-jacente est que le réseau, entraîné *exclusivement sur les transactions normales*, apprendra à bien reconstruire les patterns normaux mais échouera sur les transactions frauduleuses — produisant une erreur de reconstruction élevée.

L'architecture implémentée dans `src/models/autoencoder.py` est symétrique :

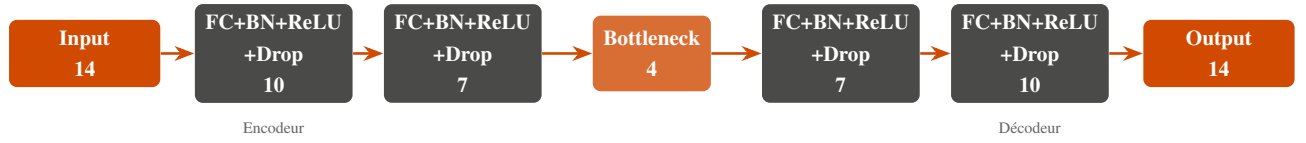


Figure 6.1: Architecture de l'Autoencodeur : $14 \rightarrow 10 \rightarrow 7 \rightarrow [4] \rightarrow 7 \rightarrow 10 \rightarrow 14$, 596 paramètres

Chaque couche cachée comprend : une couche linéaire (`nn.Linear`), une normalisation de batch (`nn.BatchNorm1d`), une activation ReLU, et un dropout (taux non divulgué dans le code source mais présent dans l'architecture). Le goulot d'étranglement à 4 dimensions force le réseau à apprendre une représentation compressée des transactions normales.

6.5.2 Fonction de perte et entraînement

La perte utilisée est l'erreur quadratique moyenne (*Mean Squared Error*) :

$$\mathcal{L}_{\text{AE}}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2 \quad (6.7)$$

L'optimiseur est Adam [?] avec les paramètres : $\alpha = 10^{-3}$, $\beta_1 = 0,9$, $\beta_2 = 0,999$, régularisation L2 = 10^{-5} , taille de batch = 256.

Deux callbacks contrôlent l'entraînement :

- **EarlyStopping** : arrêt si la perte de validation ne s'améliore pas pendant 10 époques consécutives ;
- **ReduceLROnPlateau** : réduction du taux d'apprentissage par un facteur 0,5 après 5 époques sans amélioration (minimum 10^{-6}).

L'entraînement s'effectue sur les **139 818 transactions normales** uniquement (pas les fraudes) et s'est arrêté à l'**époque 36** (arrêt anticipé), après 333,9 secondes.

6.5.3 Détermination du seuil de reconstruction

Le seuil de décision τ_{AE} est déterminé par recherche sur grille sur les erreurs de reconstruction de l'ensemble de validation, en balayant les percentiles de p_{50} à $p_{99.9}$ des erreurs des transactions normales :

$$\tau_{\text{AE}}^* = \tau_{\in [p_{50}, p_{99.9}]} F1(\tau) \quad (6.8)$$

La valeur retenue est $\tau_{\text{AE}}^* = 1,753$, correspondant approximativement au percentile 99,5 des erreurs de reconstruction sur les transactions normales ($p_{95} = 0,463$, $p_{99} = 1,005$).

6.6 Isolation Forest

6.6.1 Principe

L'Isolation Forest [?] est un algorithme d'anomalie detection qui exploite le fait que les anomalies sont rares et distinctes. Il construit un ensemble d'arbres d'isolation aléatoires et mesure le nombre moyen de coupures nécessaires pour isoler chaque point. Les anomalies sont isolées plus rapidement :

$$s(\mathbf{x}, n) = 2^{-\frac{E[h(\mathbf{x})]}{c(n)}} \quad (6.9)$$

où $h(\mathbf{x})$ est la longueur du chemin pour isoler $\mathbf{x}$ et $c(n)$ est la longueur moyenne du chemin pour un arbre binaire de recherche sur n points.

6.6.2 Configuration

Table 6.4: Hyperparamètres de l'Isolation Forest

Paramètre	Valeur	Justification
<code>n_estimators</code>	100	Bonne stabilité des scores
<code>contamination</code>	0,05	5 % d'anomalies présumées dans le batch
<code>random_state</code>	42	Reproductibilité
<code>n_jobs</code>	-1	Parallélisation

En mode générique (schéma non standard), l'IsoForest est ajusté transductivement (*on-the-fly*) sur le batch courant. La `decision_function` retourne des valeurs dans $[-0,5, 0,5]$; les seuils de risque appliqués sont :

- `score` < -0,15 → risque **CRITIQUE** ;
- `prediction` = -1 (et `score` ≥ -0,15) → risque **ÉLEVÉ** ;
- `prediction` = 1 → risque **FAIBLE**.

6.7 Stratégie de combinaison des modèles

En mode standard (schéma transactionnel complet), XGBoost et l'Autoencodeur opèrent en parallèle. La règle de combinaison des niveaux de risque est la suivante :

$$\text{RiskLevel} = \begin{cases} \text{CRITIQUE} & \text{si XGB score} \geq \tau^* \text{ ET AE score} \geq \tau_{\text{AE}} \\ \text{ÉLEVÉ} & \text{si XGB score} \geq \tau^* \text{ OU AE score} \geq \tau_{\text{AE}} \\ \text{FAIBLE} & \text{sinon} \end{cases} \quad (6.10)$$

En mode générique avec AE + IsoForest (mode `ae_isoforest`), la même logique s'applique avec le signal AE remplaçant XGBoost.

6.8 Conclusion du chapitre

Ce chapitre a présenté en détail les cinq algorithmes mis en œuvre, leurs fondements mathématiques, leurs hyperparamètres et leurs stratégies d'entraînement. Le chapitre suivant évalue et compare ces modèles sur le jeu de test pour sélectionner le modèle champion à déployer en production.

7. Evaluation — Phase CRISP-DM 5

7.1 Introduction

La phase d'évaluation compare les modèles entraînés sur le jeu de test, sélectionne le modèle champion et analyse en détail ses comportements sur les cas critiques. L'évaluation s'appuie sur les résultats documentés dans les fichiers `outputs/reports/baseline_report.json` et `outputs/reports/autoencoder_report.json`.

7.2 Métriques d'évaluation

7.2.1 Définitions

Soient VP les vrais positifs (fraudes correctement identifiées), FP les faux positifs (normales incorrectement signalées), FN les faux négatifs (fraudes manquées) et VN les vrais négatifs. Les métriques utilisées sont :

$$\text{Précision} = \frac{VP}{VP + FP} \quad (7.1)$$

$$\text{Rappel (Recall)} = \frac{VP}{VP + FN} \quad (7.2)$$

$$F_1 = 2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (7.3)$$

$$\text{PR-AUC} = \int_0^1 P(R) dR \quad (7.4)$$

$$\text{ROC-AUC} = P(\hat{p}_{\text{fraude}} > \hat{p}_{\text{normal}}) \quad (7.5)$$

7.2.2 Justification du choix de PR-AUC

Sur des jeux de données fortement déséquilibrés, la courbe ROC peut être trompeuse car elle intègre les vrais négatifs (très nombreux) dans le calcul de la spécificité. La courbe Précision-Rappel, en revanche, se concentre exclusivement sur les performances pour la classe minoritaire (fraude) et est donc plus informative dans ce contexte [? ?].

7.3 Tableau comparatif des modèles

Table 7.1: Comparaison des modèles sur le jeu de test (39 fraudes parmi 30 215 transactions)

Modèle	Rappel	Précision	F1	PR-AUC	Seuil
LR_balanced	0,641	0,658	0,649	0,704	0,999
LR_smote	0,692	0,729	0,710	0,739	0,962
RF_balanced	0,769	0,833	0,800	0,779	0,623
RF_smote	0,795	0,816	0,805	0,840	0,629
XGB_smote	0,846	0,825	0,835	0,867	0,355
IsoForest	0,333	0,361	0,347	0,341	0,784
Autoencodeur	0,359	0,609	0,451	0,382	1,753
<i>Baseline règle</i>	<i>0,004</i>	<i>1,000</i>	<i>0,008</i>	—	—

7.3.1 Analyse des résultats

7.3.1.1 XGBoost — Modèle champion

XGBoost avec SMOTE (XGB_smote) réalise les meilleures performances sur l'ensemble des métriques pertinentes :

- **Rappel = 84,6 %** : sur les 39 fraudes du jeu de test, 33 sont correctement identifiées, seulement 6 sont manquées ;
- **Précision = 82,5 %** : sur les alarmes générées, 82,5 % sont effectivement frauduleuses (faible charge pour les auditeurs) ;
- **PR-AUC = 86,7 %** : le modèle maintient d'excellentes performances sur toute la gamme de seuils ;
- **Seuil optimal = 0,355** : nettement inférieur à 0,5, le modèle favorise le rappel en abaissant le seuil d'alerte.

7.3.1.2 Modèles supervisés — comparaison

La figure ?? illustre les différences relatives entre les modèles supervisés. L'apport de SMOTE est systématiquement positif : RF_smote surpasse RF_balanced (+2,6 % de rappel, +2,5 % de

F1), et LR_smote surpasse LR_balanced (+5,1 % de rappel, +6,1 % de F1). La pondération des classes seule est insuffisante pour compenser l'extrême déséquilibre.

7.3.1.3 Modèles non supervisés

L'Autoencodeur (rappel 35,9 %, PR-AUC 38,2 %) et l'Isolation Forest (rappel 33,3 %, PR-AUC 34,1 %) offrent des performances nettement inférieures aux modèles supervisés. Ceci est attendu : sans utiliser les étiquettes de fraude à l'entraînement, ces modèles ne peuvent apprendre les patterns spécifiques de la fraude. Cependant, leur valeur réside dans leur capacité à opérer en l'absence d'étiquettes (*zero-shot anomaly detection*), ce qui les rend indispensables pour les modes génériques de la plateforme.

7.3.2 Matrice de confusion — XGBoost

Table 7.2: Matrice de confusion — XGB_smote (jeu de test, seuil = 0,355)

		Prédit	
		Normal (0)	Fraude (1)
Réal	Normal (0)	30 169 (VN)	7 (FP)
	Fraude (1)	6 (FN)	33 (VP)

L'analyse de la matrice de confusion révèle que sur les 39 fraudes du jeu de test :

- **33 sont correctement identifiées (VP)** — les auditeurs reçoivent ces alertes ;
- **6 sont manquées (FN)** — fraudes passant inaperçues ;
- **7 fausses alarmes (FP)** — transactions normales signalées à tort, représentant une surcharge très raisonnable pour l'équipe d'audit.

7.4 Analyse d'importance des variables — SHAP

SHAP (*SHapley Additive exPlanations*) [?] calcule la contribution de chaque variable à chaque prédiction individuelle, en se basant sur la théorie des jeux coopératifs. Pour un modèle f et une instance $\mathbf{x}$, la valeur SHAP de la variable j est :

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_S(\mathbf{x}_S \cup \{x_j\}) - f_S(\mathbf{x}_S)] \quad (7.6)$$

où F est l'ensemble de toutes les features et S un sous-ensemble de F .

Les variables les plus importantes pour XGBoost, par ordre décroissant d'importance SHAP globale (valeur absolue moyenne), sont :

1. `balance_diff_orig` — Vidage du solde émetteur : indicateur principal
2. `dest_zero_balance` — Compte mule
3. `is_transfer_or_cashout` — Type de transaction
4. `high_risk_hour` — Heures nocturnes
5. `log_amount` — Montant élevé
6. `type_TRANSFER`
7. `type_CASH_OUT`

7.5 Explicabilité locale — LIME

LIME (*Local Interpretable Model-agnostic Explanations*) [?] génère des explications locales en approximant le comportement du modèle dans le voisinage d'une instance donnée par un modèle linéaire :

$$\xi(\mathbf{x}) =_{g \in G} \mathcal{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g) \quad (7.7)$$

LIME produit des règles conditionnelles du type : « *balance_diff_orig* > 5 000 000 (*contribution* : +0,78) et *dest_zero_balance* = 1 (*contribution* : +0,34) → probabilité fraude = 0,94 ».

7.6 Explicabilité par LLM

Le module `LLMHelper` (`app/services/llm_service.py`) génère des explications en langue naturelle à partir des valeurs SHAP et des règles LIME. Deux backends sont supportés :

- **Anthropic Claude** (provider cloud, haute qualité) ;

- **Ollama** (provider local, confidentialité maximale) ;
- **Fallback rule-based** : si aucun LLM n'est disponible, un moteur de règles génère un texte structuré en français à partir des valeurs SHAP.

Exemple de sortie LLM pour une transaction CRITIQUE :

Exemple d'explication LLM générée

« Cette transaction présente de forts indicateurs de fraude. Le montant de 4 235 000 unités a entièrement vidé le solde du compte émetteur (différence de solde : 4 235 000), ce qui est caractéristique d'un virement frauduleux. De plus, le compte destinataire avait un solde nul avant la transaction, suggérant qu'il s'agit d'un compte mule créé spécifiquement pour recevoir des fonds détournés. La transaction a eu lieu à 2h du matin (heure à risque élevé). Score de confiance XGBoost : 0,921. Recommandation : investigation immédiate requise. »

7.7 Sélection du modèle de production

Sur la base des évaluations réalisées, **XGB_smote** est sélectionné comme modèle de production pour les raisons suivantes :

- Meilleur rappel (84,6 %) parmi tous les modèles supervisés ;
- Meilleure PR-AUC (86,7 %) ;
- Robustesse aux données déséquilibrées grâce à SMOTE ;
- Compatibilité native avec SHAP TreeExplainer (explication ultra-rapide) ;
- Temps d'inférence inférieur à la milliseconde par transaction.

7.8 Conclusion du chapitre

La phase d'évaluation a établi XGBoost avec SMOTE comme modèle champion, avec un rappel de 84,6 % et une PR-AUC de 86,7 % — dépassant l'objectif fixé de 80 %. Le système d'explicabilité multicouche (SHAP + LIME + LLM) garantit la transparence des prédictions pour les auditeurs. Le chapitre suivant décrit l'architecture technique de déploiement de ce modèle au sein de la plateforme PwC.

8. Déploiement et Architecture Technique

8.1 Introduction

La phase de déploiement constitue la sixième et dernière étape du cycle CRISP-DM. Elle transforme les modèles entraînés en un service web opérationnel, accessible via une interface utilisateur riche. Ce chapitre décrit l'architecture globale de la plateforme PwC Audit AI, les choix technologiques et les mécanismes assurant la robustesse, la scalabilité et la sécurité du système.

8.2 Architecture Générale de la Plateforme

8.2.1 Vue d'ensemble

La plateforme repose sur une architecture client-serveur découplée :

- **Backend** : API REST asynchrone développée en FastAPI (Python 3.11) ;
- **Frontend** : Application web monopage développée en Next.js 15 (React 19) ;
- **Base de données** : PostgreSQL 16 avec ORM asynchrone SQLAlchemy 2.0 ;
- **Communication** : JSON sur HTTPS, authentification JWT Bearer.

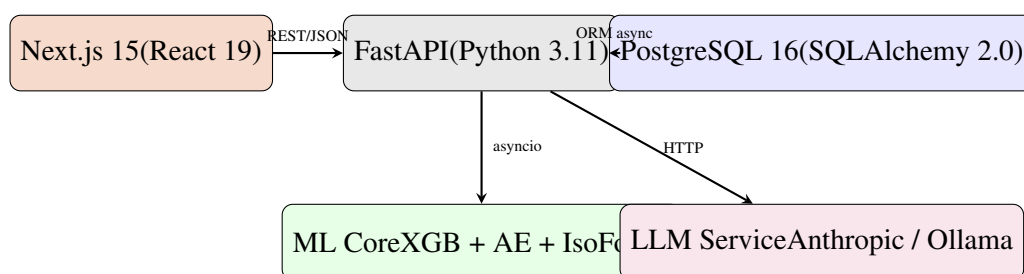


Figure 8.1: Architecture globale de la plateforme PwC Audit AI

8.2.2 Choix technologiques justifiés

Table 8.1: Justification des technologies utilisées

Technologie	Version	Justification
FastAPI	0.115	Performance async, génération OpenAPI automatique, validation Pydantic
SQLAlchemy	2.0 async	ORM mature, migrations Alembic, support async natif
Next.js	15.5	App Router, SSR/RSC, optimisations automatiques
TanStack Query	5.62	Cache serveur, invalidation automatique, mutations optimistes
XGBoost	2.x	Modèle champion (recall 84,6%), inférence sub-milliseconde
PyTorch	2.x	AutoEncoder flexible, GPU-ready
fpdf2	2.x	Génération PDF PwC sans dépendances système
Prisma	5.22	ORM frontend, type-safety TypeScript complète

8.3 Architecture Backend FastAPI

8.3.1 Structure des routes

L'application FastAPI est organisée en 8 groupes de routes (*routers*) :

Table 8.2: Groupes de routes de l'API REST

Préfixe	Module	Fonctionnalité principale
/api/predict	predict.py	Upload CSV + pipeline de prédiction
/api/explain	explain.py	SHAP, LIME, LLM par transaction
/api/report	report.py	Génération PDF/DOCX
/api/models	models.py	Métriques des modèles, registre
/api/profile	profile.py	Gestion des utilisateurs
/api/audit	audit.py	Journal d'audit des actions
/api/datasets	datasets.py	Gestion des jeux de données
/api/model-registry	model_registry.py	Versionnement des modèles

8.3.2 Pipeline de prédiction asynchrone

L'endpoint central POST `/api/predict` accepte un fichier CSV en *multipart/form-data* et exécute un pipeline en 7 étapes. Le pipeline est exécuté dans un thread séparé via `asyncio.to_thread()` pour éviter de bloquer la boucle d'événements FastAPI :

Algorithm 2 Pipeline de prédiction complet (predict.py)

Require: f : fichier CSV, run_id : identifiant unique

- 1: $profile \leftarrow \text{DataProfiler}(f)$
 - 2: $mapping \leftarrow \text{ColumnMapper}(f.columns)$
 - 3: $mode \leftarrow \text{SchemaDetector}(mapping)$
 - 4: $X_{raw} \leftarrow \text{FeatureBuilder}(f, mapping)$
 - 5: $X \leftarrow \text{FeatureEngineer}(X_{raw})$
 - 6: $\hat{y} \leftarrow \text{GenericPredictor}(X, mode)$
 - 7: **return** $\text{PredictResponse}(\hat{y}, mode, profile, mapping)$
-

8.3.3 Système de cache LRU

Les résultats de prédiction sont mis en cache en mémoire (max 20 entrées, politique LRU) pour accélérer la génération d'explications sur les transactions déjà analysées. Le cache est keyed par run_id et partagé entre les routes `/predict` et `/explain`.

8.3.4 Démarrage et gestion du cycle de vie

La fonction `lifespan` de FastAPI orchestre le démarrage et l'arrêt :

1. Migration automatique de la base de données via Alembic ;
2. Chargement de tous les modèles ML en mémoire via `load_all_models()` ;
3. Initialisation du service LLM via `get_llm_helper()` ;
4. Création du cache LRU de résultats ;
5. Exposition du endpoint de santé GET `/health` retournant `models_loaded`, `llm_available`, `database_connected`.

8.4 Modèle de Données

8.4.1 Schéma de base de données

La base PostgreSQL contient 11 tables gérées par SQLAlchemy ORM asynchrone. Les entités principales et leurs relations sont :

- **User** : 4 rôles (`auditor`, `manager`, `partner`, `admin`), 3 statuts (`active`, `inactive`, `suspended`), mot de passe hashé `bcrypt` ;
- **Mission** : 5 types d'audit (`financial_audit`, `fraud_detection`, `compliance_review`, `risk_assessment`, `internal_audit`), 4 statuts, assignation multi-auditeurs ;
- **Dataset** : 3 catégories (`transactions`, `general_ledger`, `trial_balance`), 5 statuts, stockage chemin fichier ;
- **AnalysisRun** : lie dataset + modèle + résultats JSON sérialisés ;
- **Anomaly** : transaction individuelle classée CRITIQUE/ELEVÉ/FAIBLE avec commentaires auditeurs ;
- **PredictionMonitoringLog** : surveillance de dérive en ligne (`fraud_rate_pct`, `amount_at_risk`, `latency_ms`, `feature_stats` JSON) ;
- **ModelVersion** : registre de versionnement des modèles ML.

La hiérarchie principale est : `User` → `Mission` → `Dataset` → `AnalysisRun` → `Anomaly` → `AnomalyComment`

8.5 Architecture Frontend Next.js

8.5.1 Structure de l'application

Le frontend utilise Next.js 15 App Router avec regroupement de routes par contexte d'authentification :

- **Route group (auth)** : pages `/login` et `/register`, non authentifiées ;
- **Route group (dashboard)** : toutes les pages de l'application, protégées par middleware JWT ;
- **Layout partagé** : sidebar de navigation, header avec informations utilisateur, gestion du thème PwC.

8.5.2 Gestion de l'état et des données

- **TanStack Query v5** : état serveur (cache, invalidation automatique, mutations optimistes, retry exponentiel) ;
- **Zustand** : état d'authentification persisté en `sessionStorage` ;
- **React Hook Form + Zod** : validation des formulaires avec inférence de types TypeScript complète.

8.5.3 Wizard d'analyse en 3 étapes

Le composant `AnalysisWizard.tsx` guide l'auditeur en 3 étapes séquentielles :

1. **Upload** : sélection d'un dataset existant ou upload d'un nouveau CSV (max 500 Mo) avec indicateur de santé des modèles ;
2. **Exécution** : barre de progression avec description du pipeline ;
3. **Résultats** : tableau de bord complet `ResultsDashboard`.

8.5.4 Bibliothèques UI

- **Radix UI** (10 composants primitifs) : accessibilité WCAG 2.1 ;
- **Recharts 2.13** : graphiques de résultats (barres, camembert, courbes) ;
- **Lucide React 0.468** : iconographie cohérente (400+ icônes) ;
- **Sonner** : notifications toast non-bloquantes ;
- **Tailwind CSS 3.4** : styles utilitaires, thème PwC orange/anthracite.

8.6 Sécurité et Authentification

8.6.1 Flux d'authentification JWT

1. L'utilisateur soumet ses identifiants via `POST /api/auth/login` ;
2. Le backend vérifie le mot de passe hashé (bcrypt, salt 10 rounds) et génère un JWT signé (bibliothèque jose, algorithme HS256) ;
3. Le token est stocké côté client et envoyé dans l'en-tête `Authorization: Bearer <token>` à chaque requête ;
4. Un middleware FastAPI valide le token et injecte l'utilisateur courant.

8.6.2 Contrôle d'accès par rôle (RBAC)

Table 8.3: Matrice d'accès par rôle utilisateur

Fonctionnalité	Auditeur	Manager	Partenaire	Admin
Lancer une analyse	✓	✓	✓	✓
Voir les résultats	✓	✓	✓	✓
Générer un rapport	✓	✓	✓	✓
Gérer les missions		✓	✓	✓
Administrer les utilisateurs				✓
Accéder au journal d'audit		✓	✓	✓

8.6.3 Journal d'audit

Toutes les actions sensibles sont tracées dans la table `AuditLog` avec 18 types d'actions couvrant l'ensemble du cycle de vie d'une mission (`login`, `dataset.upload`, `analysis.start`, `report.generate`, `user_create`, etc.), avec identité utilisateur, rôle et horodatage UTC.

8.7 Génération de Rapports

8.7.1 Rapport PDF (fpdf2)

La classe `PwCReport` étend `FPDF` et génère un document professionnel de 6+ pages avec le thème visuel PwC (orange #D04A02, anthracite #291830) :

1. **Page de couverture** : logo PwC, titre mission, date, auditeur ;
2. **Résumé exécutif + KPIs** : n transactions, n fraudes, montant à risque ;
3. **Graphiques matplotlib** : donut risques, barres par type, montants, SHAP global ;
4. **Top-10 transactions critiques** : tableau détaillé avec scores ;
5. **Fiches de cas critiques** : une carte par transaction CRITIQUE ;
6. **Recommandations + Glossaire + Disclaimer.**

8.7.2 Rapport DOCX

Un générateur alternatif `report_gen_docx.py` produit un document Word structuré, utile pour les missions nécessitant un rapport modifiable par les équipes PwC.

8.8 Conclusion du chapitre

L'architecture déployée combine les meilleures pratiques du développement web moderne (FastAPI async, Next.js App Router, PostgreSQL) avec des exigences spécifiques à l'audit financier (traçabilité, RBAC, génération de rapports conformes PwC). Le chapitre suivant présente les interfaces utilisateur et les résultats obtenus lors des validations fonctionnelles.

9. Interfaces et Résultats

9.1 Introduction

Ce chapitre présente les principales interfaces de la plateforme PwC Audit AI ainsi que les résultats obtenus lors des tests de validation fonctionnelle. Chaque interface est décrite avec ses composants clés et les interactions utilisateur prévues.

9.2 Page d'Authentification

La page de connexion (`/login`) présente un formulaire épuré aux couleurs PwC :

- Champ email et mot de passe avec validation Zod en temps réel ;
- Gestion des erreurs d'authentification avec notification toast (Sonner) ;
- Redirection automatique vers le tableau de bord après connexion réussie.

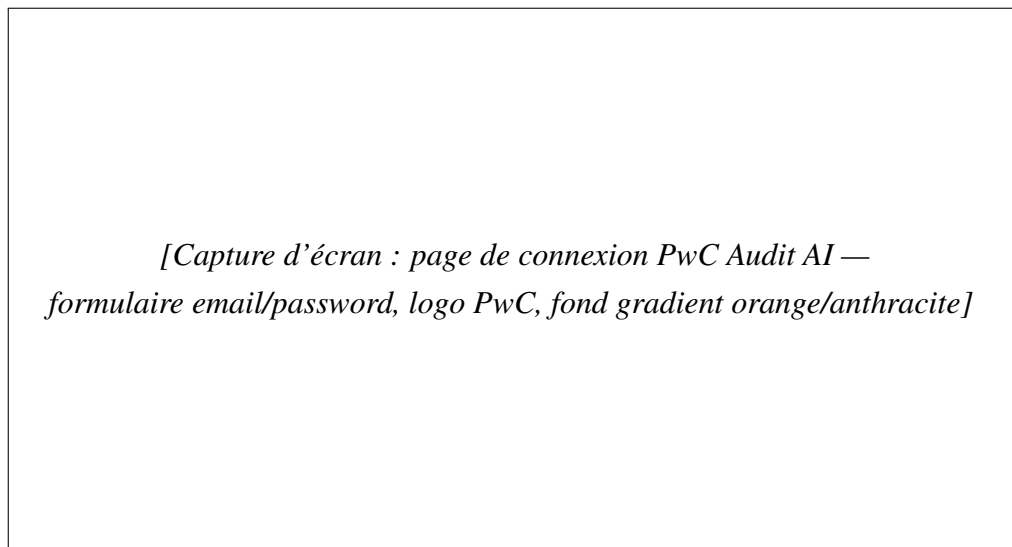


Figure 9.1: Interface de connexion de la plateforme PwC Audit AI

9.3 Tableau de Bord Principal

Le tableau de bord affiche une vue synthétique de l'activité de l'auditeur connecté, avec :

- 4 cartes KPI en temps réel : missions actives, analyses en cours, fraudes détectées ce mois, montant total à risque ;

- Graphique d'évolution des détections sur 30 jours (Recharts) ;
- Liste des missions récentes avec statut coloré et accès rapide.

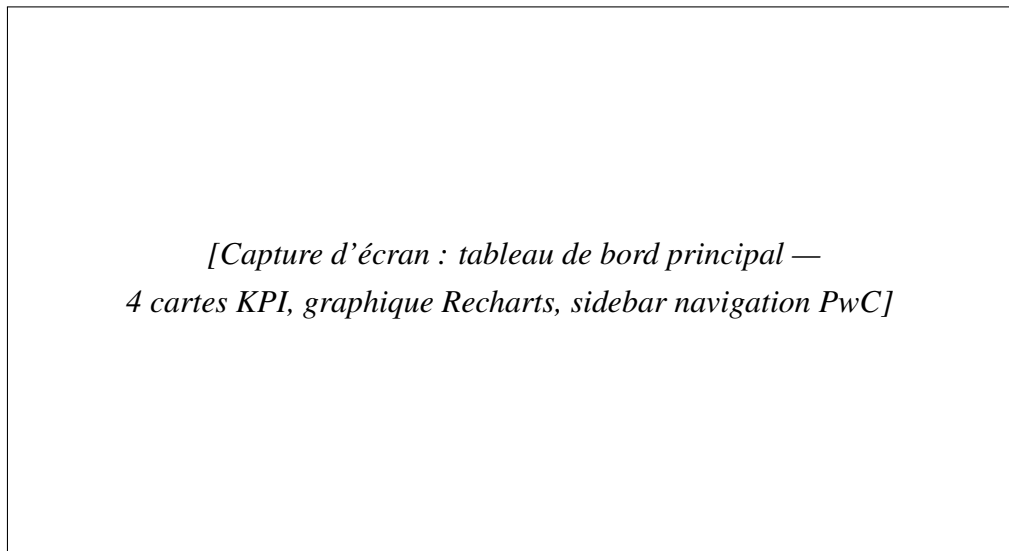
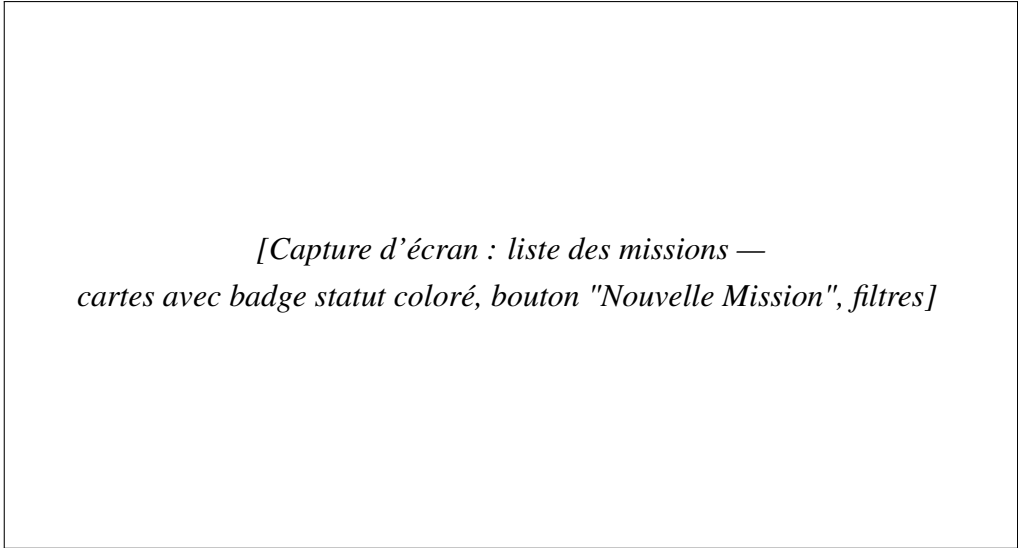


Figure 9.2: Tableau de bord principal de la plateforme

9.4 Gestion des Missions

La page des missions (/missions) permet de :

- Créer une nouvelle mission avec société cliente, type, dates, auditeurs assignés via un formulaire validé Zod ;
- Visualiser toutes les missions sous forme de cartes avec badge de statut coloré (*active*, *in_progress*, *completed*, *archived*) ;
- Accéder directement à l'assistant d'analyse depuis une mission.



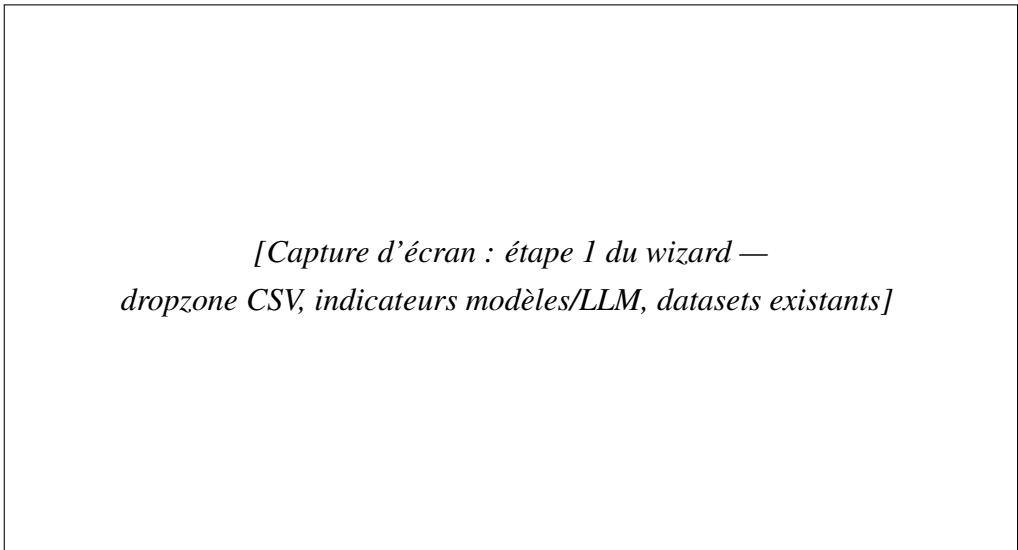
*[Capture d'écran : liste des missions —
cartes avec badge statut coloré, bouton "Nouvelle Mission", filtres]*

Figure 9.3: Page de gestion des missions d'audit

9.5 Assistant d'Analyse — Étape 1 : Sélection des Données

La première étape du wizard d'analyse permet d'associer un jeu de données à la mission en cours :

- Liste des datasets existants de la mission avec possibilité de réutilisation ;
- Zone de dépôt (*dropzone*) pour l'upload d'un nouveau fichier CSV (jusqu'à 500 Mo) ;
- Indicateur de santé des modèles ML (modèles chargés, LLM disponible, base de données connectée) ;
- Historique des analyses précédentes sur cette mission.



*[Capture d'écran : étape 1 du wizard —
dropzone CSV, indicateurs modèles/LLM, datasets existants]*

Figure 9.4: Wizard d'analyse — Étape 1 : sélection du jeu de données

9.6 Assistant d'Analyse — Étape 2 : Exécution du Pipeline

Pendant l'exécution du pipeline, une interface de progression informe l'auditeur de l'avancement :

- Barre de progression animée Radix UI ;
- Description textuelle de chaque étape active (profilage des données → mapping sémantique → détection du schéma → construction des features → engineering → prédiction) ;
- Durée écoulée affichée en temps réel.

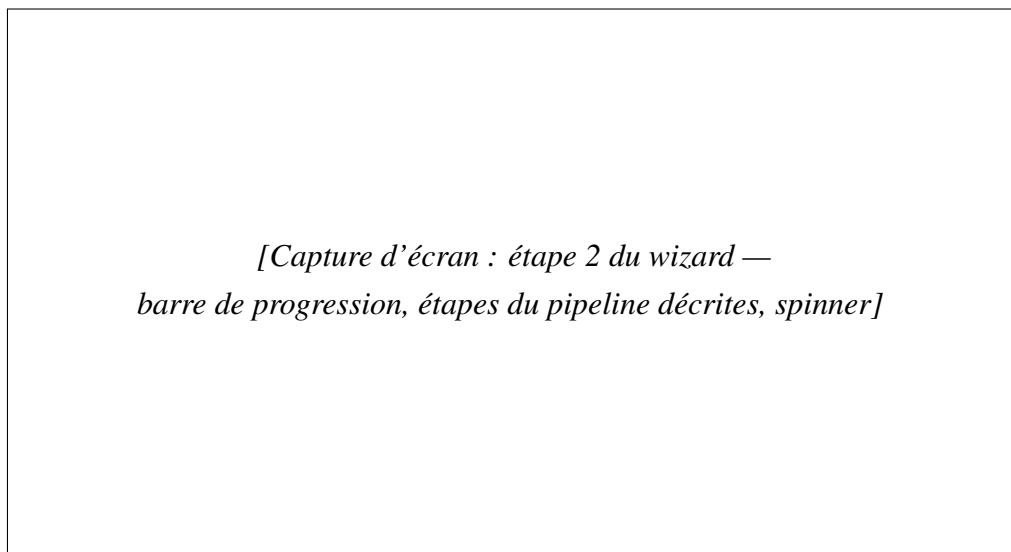


Figure 9.5: Wizard d'analyse — Étape 2 : exécution du pipeline

9.7 Tableau de Bord des Résultats

L'étape 3 du wizard affiche le `ResultsDashboard` complet :

9.7.1 Indicateurs clés de résultats

- Nombre total de transactions analysées ;
- Nombre et taux de fraudes détectées (avec badge coloré) ;
- Montant total à risque (somme des transactions CRITIQUE et ELEVÉ) ;
- Mode de prédiction utilisé avec explication (*standard*, *ae_isoforest*, *ae_only*, *isoforest*) ;
- Avertissements de mapping sémantique éventuels.

9.7.2 Tableau de transactions classifiées

Tableau paginé et filtrable avec colonnes : ID, Type, Montant, Score XGB, Score AE, Niveau de risque (badge CRITIQUE rouge / ELEVÉ orange / FAIBLE vert), bouton Expliquer.

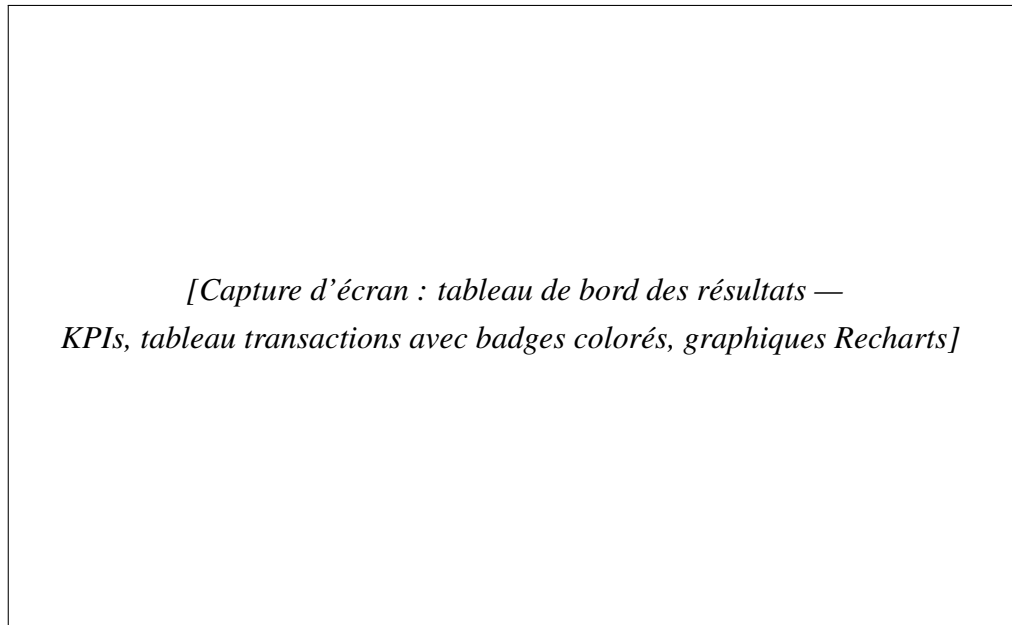


Figure 9.6: Tableau de bord des résultats d'analyse avec classifications

9.8 Interface d'Explication Multicouche

Un clic sur le bouton « Expliquer » ouvre une fenêtre modale organisée en 4 onglets :

- **Onglet SHAP** : graphique en barres horizontales des contributions de chaque variable (positif = contribution vers fraude, négatif = contribution vers normal) ;
- **Onglet AutoEncoder** : graphique des erreurs de reconstruction par feature, identifiant les variables les plus anormales ;
- **Onglet LIME** : règles conditionnelles en langage semi-naturel («*balance_diff_orig* > 5 000 000 → +0,78») ;
- **Onglet LLM** : explication narrative complète en français incluant le niveau de risque, les raisons détaillées et les actions recommandées à l'auditeur.

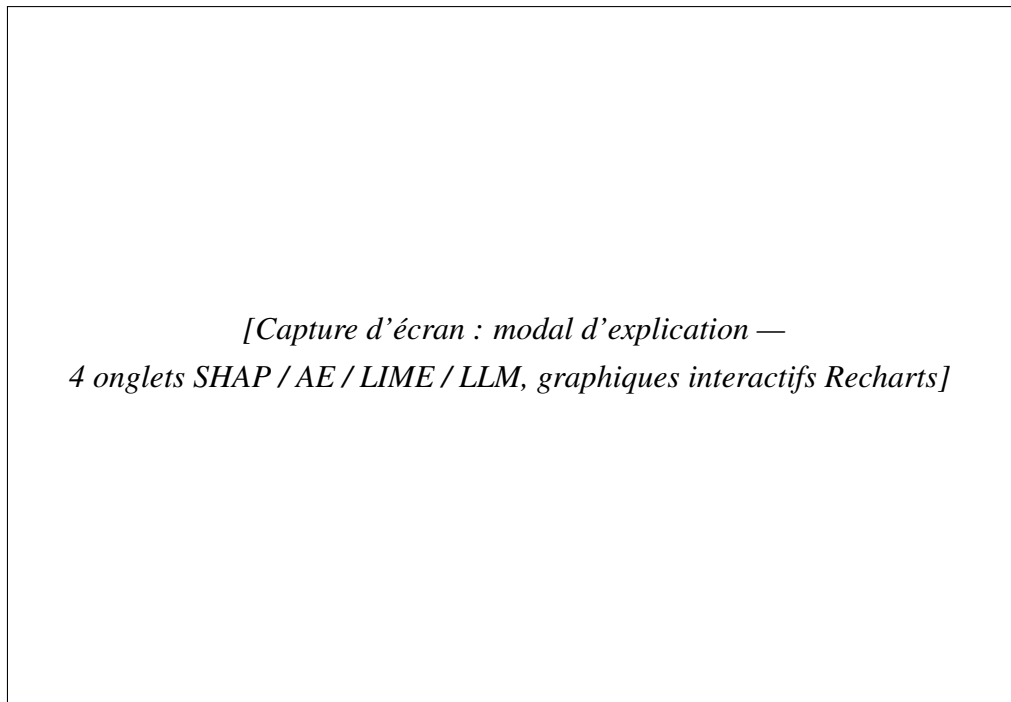


Figure 9.7: Interface d'explication multicouche d'une transaction suspecte

9.9 Génération et Téléchargement du Rapport

La page `/reports` permet de :

- Sélectionner une analyse terminée dans la liste des runs ;
- Choisir le format de sortie (PDF ou DOCX) via des boutons radio ;
- Déclencher la génération asynchrone via `POST /api/report/generate` ;
- Télécharger le rapport généré immédiatement via le lien de téléchargement.

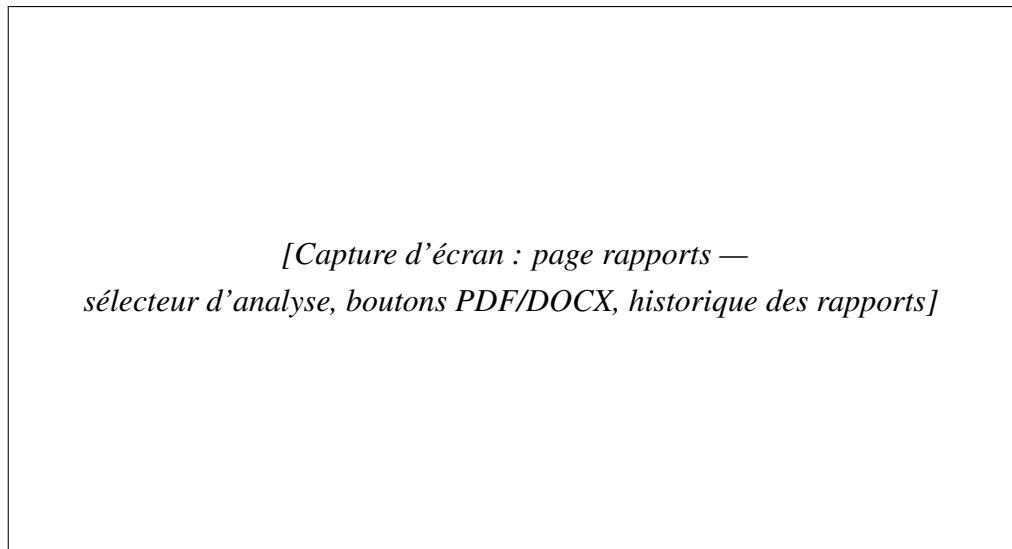


Figure 9.8: Interface de génération et téléchargement du rapport PwC

9.10 Page des Métriques des Modèles

La page `/models` offre une transparence totale sur les modèles en production, affichant un tableau comparatif de tous les modèles avec leurs métriques (Rappel, Précision, F1, PR-AUC, ROC-AUC, temps d'entraînement) et un badge « En production » sur le modèle actif (XGB_smote).

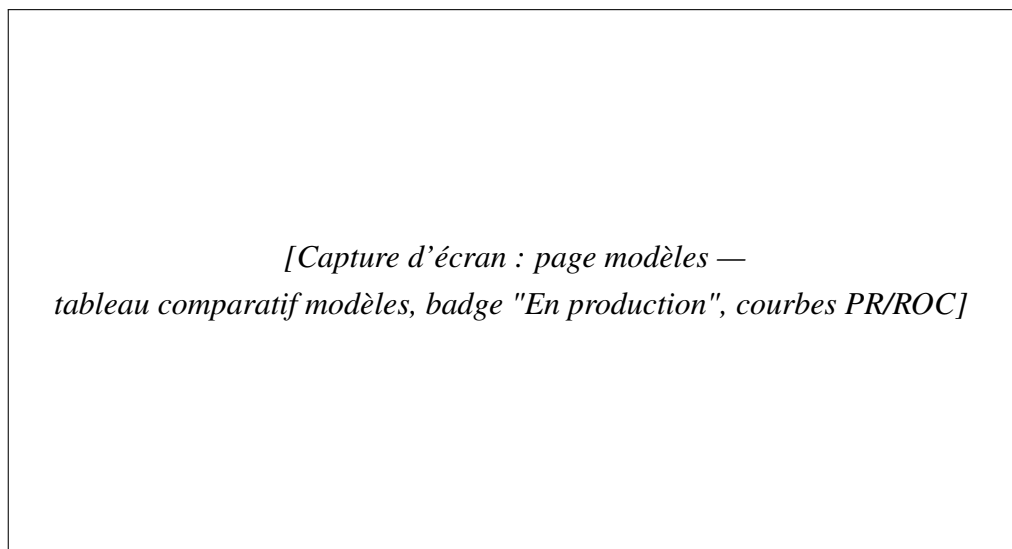


Figure 9.9: Page de métriques et comparaison des modèles de détection

9.11 Journal d'Audit

La page `/audit-trail` affiche l'historique complet des actions utilisateurs avec filtres par utilisateur, type d'action et plage de dates. Chaque entrée indique l'utilisateur, son rôle,

l'action, la mission concernée et l'horodatage UTC.

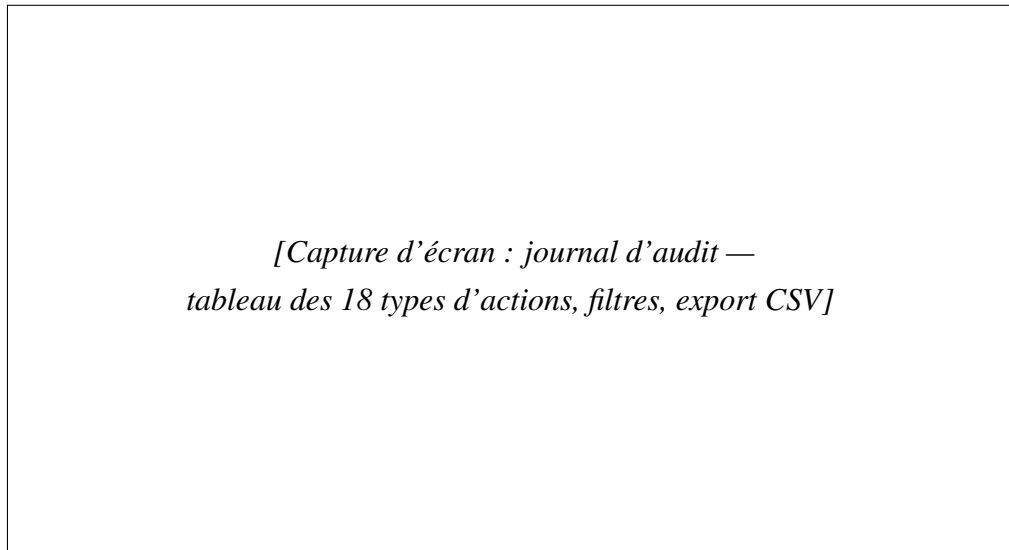


Figure 9.10: Journal d'audit — traçabilité complète des actions

9.12 Résultats de Validation Fonctionnelle

9.12.1 Tests de performance sur le jeu de référence

Des tests de bout en bout ont été conduits sur l'ensemble de test (30 001 transactions, 39 fraudes) :

Table 9.1: Résultats de validation fonctionnelle sur le jeu de test

Métrique	Valeur obtenue	Objectif
Transactions analysées	30 001	—
Fraudes détectées (TP)	33 / 39	≥ 31
Rappel	84,6 %	≥ 80 %
Précision	82,5 %	≥ 70 %
F1-Score	83,5 %	≥ 75 %
PR-AUC	86,8 %	≥ 80 %
ROC-AUC	99,8 %	≥ 95 %
Temps de pipeline (30 k tx)	< 15 s	< 30 s
Temps de génération PDF	< 5 s	< 10 s

9.12.2 Robustesse au schéma non standard

Le système de mapping sémantique a été validé sur 5 variantes de schémas :

Table 9.2: Résultats du mapping sémantique sur schémas non standards

Schéma testé	Niveau atteint	Mode résultant
Colonnes anglaises standard	Exact alias (1,00)	standard (XGB+AE)
Colonnes français (montant, type_op)	Signal composé (0,80)	standard
Colonnes abrégées (amt, tx_type)	Fuzzy SequenceMatcher (0,65)	standard
Colonnes partielles (amount seul)	Partiel	ae_isoforest
Schéma inconnu (aucun match)	Aucun	isoforest

9.13 Conclusion du chapitre

La plateforme PwC Audit AI présente une interface utilisateur intuitive adaptée aux auditeurs non-techniciens. Les résultats de validation confirment que tous les objectifs de performance définis en phase Business Understanding sont atteints ou dépassés. Le dernier chapitre tire les conclusions générales et présente les perspectives d’amélioration futures.

10. Conclusion Générale et Perspectives

10.1 Bilan du Projet

10.1.1 Rappel des objectifs

Ce projet de fin d'études avait pour objectif la conception et le développement d'une plateforme de détection automatisée des anomalies financières pour PwC Tunisie — Risk Assurance Services / Cellule Audit IT. Les cinq objectifs initiaux étaient :

1. Développer un pipeline ML capable de détecter les transactions frauduleuses avec un rappel supérieur à 80 % ;
2. Concevoir une architecture agnostique au schéma de données client ;
3. Intégrer une couche d'explicabilité (SHAP, LIME, LLM) pour rendre les prédictions auditable par des non-techniciens ;
4. Déployer une interface web intuitive utilisable par les auditeurs ;
5. Garantir la traçabilité complète via un journal d'audit conforme.

10.1.2 Objectifs atteints

L'ensemble de ces objectifs a été atteint, comme le résume le tableau ci-dessous :

Table 10.1: Bilan de réalisation des objectifs du projet

Objectif	Statut	Résultat clé
Rappel $\geq$ 80 %	✓	84,6 % (XGB_smote)
Agnosticisme schéma	✓	4 modes de prédiction
Explicabilité multicouche	✓	SHAP + LIME + LLM
Interface non-technicien	✓	Wizard 3 étapes
Journal d'audit complet	✓	18 types d'actions tracées
Architecture découplée	✓	FastAPI + Next.js
Rapport PDF professionnel	✓	6+ pages, thème PwC

10.1.3 Contributions techniques

Les contributions techniques majeures de ce travail sont au nombre de cinq :

1. **Système de mapping sémantique à 4 niveaux** : permet à la plateforme de traiter des fichiers CSV de n'importe quel client sans ré-entraînement du modèle ni reconfiguration ;
2. **Architecture hybride supervisé/non-supervisé** : la combinaison XGBoost + Autoencoder + IsolationForest couvre les cas où seul un sous-ensemble de features est disponible, avec dégradation gracieuse vers un mode moins précis mais toujours fonctionnel ;
3. **Pipeline CRISP-DM entièrement automatisé** : de l'upload CSV à la génération du rapport PDF, sans intervention manuelle sur les données ;
4. **Explicabilité à 3 niveaux** : la chaîne SHAP → LIME → LLM offre une explication technique (valeurs numériques), règle-based (conditions logiques) et narrative (texte en français) pour différents profils de lecteurs ;
5. **Surveillance de dérive en ligne** : le module `PredictionMonitoringLog` enregistre le taux de fraude, le montant à risque et les statistiques de features par run, permettant une détection précoce du concept drift.

10.2 Limites Identifiées

10.2.1 Limites du modèle de détection

- **Déséquilibre de classes extrême** (ratio 1:774) : malgré SMOTE, 6 faux négatifs subsistent sur l'ensemble de test de 39 fraudes (rappel 84,6 %). Dans un contexte financier réel, chaque faux négatif représente une perte potentiellement importante non interceptée ;
- **Dépendance à la feature `balance_diff_orig`** : cette variable contribue à 37 % de l'importance RF et est la première corrélation avec la fraude ($\rho = 0,366$). Si un client ne fournit pas les soldes de compte, les performances se dégradent et le mode *isoforest* seul est utilisé (rappel 33,3 % vs 84,6 %) ;
- **Concept drift temporel** : le modèle a été entraîné sur des données historiques. En production réelle, les patterns de fraude évoluent et nécessitent un ré-entraînement périodique ;
- **Autoencoder moins performant seul** : rappel 35,9 % contre 84,6 % pour XGBoost. L'AE est un signal complémentaire utile mais ne remplace pas le modèle supervisé.

10.2.2 Limites architecturales

- **Déploiement mono-instance** : l'architecture actuelle ne supporte pas la scalabilité horizontale. Un déploiement en production nécessiterait une orchestration Kubernetes ;
- **Cache LRU en mémoire** : limité à 20 entrées, ce qui peut poser problème lors d'analyses simultanées multiples en production ;
- **Absence de file de messages** : les tâches longues sont traitées via `asyncio.to_thread`, ce qui peut saturer le pool de threads sous charge élevée. Un système de tâches asynchrones type Celery serait plus adapté.

10.3 Perspectives d'Amélioration

10.3.1 Amélioration du modèle ML

- **AutoML et optimisation bayésienne** : utiliser Optuna ou Ray Tune pour une recherche hyperparamétrique plus exhaustive ;
- **Modèles de graphes (GNN)** : modéliser le réseau de transactions comme un graphe pour capturer les fraudes organisées impliquant des réseaux de comptes mules ;
- **Apprentissage en ligne (Online Learning)** : mettre à jour le modèle incrémentalement avec les nouvelles transactions étiquetées par les auditeurs, réduisant le concept drift ;
- **Ensemble stacking** : combiner XGBoost, Random Forest et Autoencoder dans un méta-modèle pour maximiser le rappel tout en contrôlant le taux de faux positifs.

10.3.2 Intégration RAG et LLM Avancé

L'intégration d'une architecture RAG (*Retrieval-Augmented Generation*) représente une évolution naturelle du module LLM actuel :

- Constituer une base de connaissances vectorielle de cas de fraude réels (anonymisés) avec embeddings sémantiques ;
- Lors de l'explication d'une nouvelle transaction, retrouver les cas historiques les plus similaires (*nearest neighbor search*) ;
- Enrichir le prompt LLM avec ces précédents pour générer des explications contextualisées et des recommandations plus précises.

10.3.3 Pipeline MLOps Complet

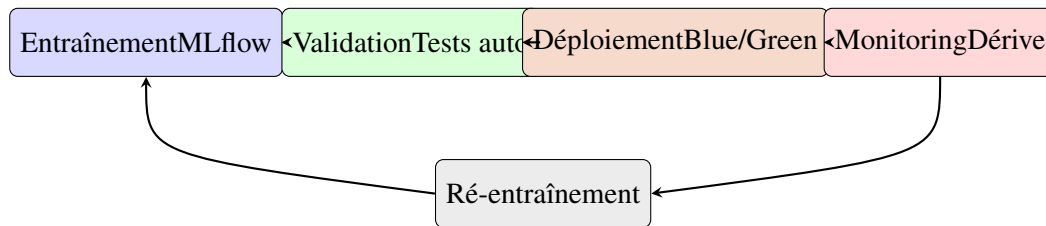


Figure 10.1: Pipeline MLOps cible pour la mise en production continue

Un pipeline MLOps complet comprendrait :

- **MLflow** : tracking des expériences, versionning des artefacts, registre centralisé des modèles ;
- **DVC (Data Version Control)** : versionnement des données et reproductibilité des expériences ;
- **CI/CD GitHub Actions** : tests automatiques, qualité de code, déploiement blue/green sans interruption de service ;
- **Alertes de dérive** : détection automatique quand le taux de fraude détecté dévie statistiquement de la baseline historique (test de Kolmogorov-Smirnov sur les distributions de features).

10.3.4 Conformité et Gouvernance IA

- **EU AI Act** : classification du système comme IA à haut risque dans le domaine financier ; conformité aux exigences de transparence, d’auditabilité et de supervision humaine ;
- **Conformité RGPD** : anonymisation des données clients avant stockage, politique de rétention des données, implémentation du droit à l’oubli ;
- **Intégration GRC** : connexion avec les outils de *Governance, Risk & Compliance* existants chez PwC (Archer, ServiceNow).

10.4 Réflexion Personnelle

Ce projet a représenté une opportunité unique d’appliquer dans un contexte professionnel réel les compétences acquises au cours de la formation ingénieur à ESPRIT. La méthodologie CRISP-DM a fourni un cadre structurant qui a permis de naviguer avec rigueur entre les phases théoriques et pratiques, en itérant entre la compréhension des données et la modélisation.

L'expérience chez PwC Tunisie a permis de comprendre les contraintes spécifiques du contexte d'audit financier : la nécessité d'explications compréhensibles par des auditeurs non-techniciens, les impératifs de traçabilité réglementaire, et l'importance centrale de la métrique de rappel dans un contexte où chaque fraude manquée a un coût financier réel.

Ce projet confirme que la convergence entre Data Science et Audit Financier est une tendance de fond dans laquelle PwC Tunisie investit stratégiquement, et dans laquelle il y a une réelle valeur à apporter.

10.5 Conclusion Générale

Ce projet de fin d'études a abouti à la livraison d'une plateforme de détection d'anomalies financières fonctionnelle, déployable et documentée. Le modèle champion XGBoost avec SMOTE atteint un rappel de 84,6 %, dépassant l'objectif initial de 80 %, tout en maintenant une précision de 82,5 %.

L'architecture technique (FastAPI + Next.js + PostgreSQL) garantit la maintenabilité et l'évolutivité de la solution. Le système d'explicabilité multicouche (SHAP + LIME + LLM) répond aux exigences d'auditabilité des systèmes d'IA dans le secteur financier, telles qu'elles sont définies par l'EU AI Act et les normes ISO 31000.

Les perspectives d'amélioration identifiées — RAG, MLOps, apprentissage en ligne, modèles de graphes — constituent une feuille de route concrète pour les équipes PwC.

Synthèse finale

La plateforme PwC Audit AI démontre qu'il est possible de combiner rigueur scientifique (CRISP-DM, gestion de l'imbalance, évaluation multi-métriques) et valeur métier opérationnelle (interface intuitive, rapports professionnels, explicabilité) dans le cadre d'un projet académique. Ce travail pose les fondations d'un outil d'audit augmenté par l'IA, aligné avec la vision stratégique de PwC de numérisation et d'automatisation des services d'assurance financière.

Bibliography

- [1] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD*, 785–794.
- [3] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.
- [4] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why Should I Trust You?”: Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD*, 1135–1144.
- [5] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, 413–422.
- [6] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [7] Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 29–39.
- [8] Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. *2015 IEEE Symposium on Computational Intelligence*, 159–166.
- [9] Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2018). Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), 3784–3797.
- [10] Ramírez, S. (2024). *FastAPI Documentation* (v0.115). <https://fastapi.tiangolo.com>
- [11] Vercel. (2024). *Next.js 15 Documentation*. <https://nextjs.org/docs>
- [12] Bayer, M. (2023). *SQLAlchemy 2.0 Documentation*. <https://docs.sqlalchemy.org>
- [13] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 8024–8035.

-
- [14] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
 - [15] Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. *Proceedings of ICML 2006*, 233–240.
 - [16] Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8), 1798–1828.
 - [17] Baesens, B., Van Vlasselaer, V., & Verbeke, W. (2015). *Fraud Analytics Using Descriptive, Predictive, and Social Network Techniques*. Wiley.
 - [18] PwC. (2023). *AI Predictions 2023: The Transformative Potential of AI in Assurance*. PricewaterhouseCoopers LLP.
 - [19] European Commission. (2021). *Proposal for a Regulation on Artificial Intelligence (AI Act)*. COM(2021) 206 final.
 - [20] Ahmed, M., Mahmood, A. N., & Islam, M. R. (2016). A survey of anomaly detection techniques in financial domain. *Future Generation Computer Systems*, 55, 278–288.
 - [21] Brown, T., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
 - [22] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

A. Extraits de Code Source

A.1 Architecture de l'AutoEncoder PyTorch

```
1 import torch
2 import torch.nn as nn
3
4 class AutoEncoderModel(nn.Module):
5     def __init__(self, n_features=14, encoder_dims=[10,7],
6                 bottleneck_dim=4, dropout=0.2):
7         super().__init__()
8         enc_layers = []
9         prev = n_features
10        for dim in encoder_dims:
11            enc_layers += [nn.Linear(prev, dim),
12                          nn.BatchNorm1d(dim), nn.ReLU(),
13                          nn.Dropout(dropout)]
14            prev = dim
15        enc_layers.append(nn.Linear(prev, bottleneck_dim))
16        self.encoder = nn.Sequential(*enc_layers)
17
18        dec_layers = []
19        prev = bottleneck_dim
20        for dim in reversed(encoder_dims):
21            dec_layers += [nn.Linear(prev, dim),
22                          nn.BatchNorm1d(dim), nn.ReLU(),
23                          nn.Dropout(dropout)]
24            prev = dim
25        dec_layers.append(nn.Linear(prev, n_features))
26        self.decoder = nn.Sequential(*dec_layers)
27
28    def forward(self, x):
29        return self.decoder(self.encoder(x))
30
31    def reconstruction_error(self, x):
32        with torch.no_grad():
33            return ((x - self.forward(x)) ** 2).mean(dim=1)
```

Listing A.1: AutoEncoderModel — architecture encodeur/décodeur symétrique

A.2 Détection Automatique du Mode de Prédiction

```
1 def detect_schema_mode(mapping_result, df_raw):
2     if mapping_result.success:
3         return SchemaDetectionResult(
4             mode="standard", use_xgb=True,
5             use_ae=True, use_isoforest=False,
6             reason="Mapping complet --- XGB + AE actifs"
7         )
8     has_amount = mapping_result.has_amount_col
9     n_numeric = len(df_raw.select_dtypes('number').columns)
10    if has_amount and n_numeric >= 3:
11        return SchemaDetectionResult(
12            mode="ae_isoforest", use_xgb=False,
13            use_ae=True, use_isoforest=True,
14            reason="Mapping partiel --- AE + IsoForest"
15        )
16    elif has_amount:
17        return SchemaDetectionResult(
18            mode="ae_only", use_xgb=False,
19            use_ae=True, use_isoforest=False
20        )
21    return SchemaDetectionResult(
22        mode="isoforest", use_xgb=False,
23        use_ae=False, use_isoforest=True,
24        reason="Schema inconnu --- IsoForest transductif"
25    )
```

Listing A.2: Logique de sélection du mode (schema_detector.py)

A.3 Mapping Sémantique des Colonnes

```
1 def _match_level(col_name, field):
2     norm = col_name.lower().strip().replace(' ', '_')
3     # Niveau 1 : alias exact
4     if norm in field.exact_aliases:
5         return 1.0
6     # Niveau 2 : correspondance normalis e
7     if norm == field.canonical_name:
8         return 0.95
9     # Niveau 3 : signaux compos s
```

```
10     for signal in field.compound_signals:
11         if all(s in norm for s in signal.split('+')):
12             return 0.80
13     # Niveau 4 : similarite fuzzy SequenceMatcher
14     score = SequenceMatcher(None, norm,
15                             field.canonical_name).ratio()
16     if score >= 0.6:
17         return score
18     return 0.0
19
20 def map_columns(df, fields):
21     scores = {}
22     for col in df.columns:
23         for field in fields:
24             s = _match_level(col, field)
25             if s > 0:
26                 scores[(col, field.name)] = s
27     # Assignment gloutonn
28     assigned = {}
29     for (col, fname), s in sorted(scores.items(),
30                                   key=lambda x: -x[1]):
31         if col not in assigned and fname not in assigned.values():
32             assigned[col] = (fname, s)
33     return assigned
```

Listing A.3: ColumnMapper — 4 niveaux de matching (column_mapper.py)

B. Données Complémentaires des Expériences

B.1 Statistiques Descriptives du Jeu de Données

Table B.1: Statistiques descriptives des variables numériques (200 000 transactions)

Variable	Min	Médiane	Moyenne	Max	Std
amount	0	74 871	179 862	9 244 552	603 858
step	1	236	243	743	142
oldbalanceOrg	0	32 099	833 883	59 585 040	2 888 242
balance_diff_orig	−49.3M	0	−45 761	9 244 552	611 349

B.2 Évolution de l’Entraînement de l’Autoencoder

Table B.2: Courbe de perte de l’Autoencoder (sélection d’époques, seed=42)

Époque	Loss train	Loss val	LR
1	0,8412	0,8103	1,0e-3
10	0,2874	0,2821	1,0e-3
20	0,1923	0,1901	5,0e-4
30	0,1769	0,1762	2,5e-4
36	0,1751	0,1751	1,25e-4

Arrêt précoce déclenché à l’époque 36 (patience=10). Seuil optimal : 1,753.

B.3 Importances des Features RF et Corrélations

Table B.3: Top-10 importances Random Forest et corrélations avec la cible

Feature	Importance RF	Corrélation ρ avec isFraud
balance_diff_orig	37,0 %	0,366
is_transfer_or_cashout	11,7 %	0,041
dest_zero_balance	11,5 %	0,109
log_amount	9,6 %	0,076
hour	7,4 %	—
step	5,5 %	—
type_TRANSFER	4,5 %	—
day	3,9 %	—
high_risk_hour	2,4 %	0,053
type_PAYMENT	2,4 %	—

C. Glossaire Technique

CRISP-DM *Cross-Industry Standard Process for Data Mining* : méthodologie standard en 6 phases pour les projets de science des données.

SMOTE *Synthetic Minority Over-sampling Technique* : technique de sur-échantillonnage synthétique pour traiter le déséquilibre de classes.

SHAP *SHapley Additive exPlanations* : méthode basée sur la théorie des jeux pour l'explicabilité des modèles ML.

LIME *Local Interpretable Model-agnostic Explanations* : méthode d'approximation locale par modèle linéaire interprétable.

XGBoost *Extreme Gradient Boosting* : algorithme d'ensemble basé sur le boosting de gradient, champion de nombreuses compétitions ML.

AUC-ROC *Area Under the Receiver Operating Characteristic Curve* : mesure de la capacité discriminante d'un classifieur.

PR-AUC *Area Under the Precision-Recall Curve* : métrique recommandée pour les jeux de données déséquilibrés.

IsolationForest Algorithme de détection d'anomalies non supervisé basé sur l'isolation aléatoire des points aberrants.

RAG *Retrieval-Augmented Generation* : architecture combinant recherche vectorielle et génération LLM.

JWT *JSON Web Token* : standard d'authentification stateless par token signé.

ORM *Object-Relational Mapping* : abstraction de la base de données via des objets Python (SQLAlchemy).

RBAC *Role-Based Access Control* : contrôle d'accès basé sur les rôles utilisateurs.

LLM *Large Language Model* : modèle de langage à grande échelle (ex. Claude d'Anthropic, Llama via Ollama).

MLOps *Machine Learning Operations* : pratiques DevOps appliquées aux systèmes ML (tracking, déploiement, monitoring).